

SYLLABUS

Introduction

- I. Selenium 3 vs Selenium 4
- II. Java project vs Maven Project
- III. CRM introduction
- IV. V-Tiger Test scripts
- V. Group formation

1. Data Driven Testing

- I. Properties File
- II. Excel File

2. Generic Utility

- I. Java Utility
- II. File Utility
 - i. PropertiesFile Utility
 - ii. ExcelFile Utility
- III. WebDriver Utility

3. POM

4. TestNG

- I. Introduction
- II. Basic Annotation
- III. Assertion
- IV. Helper attributes
- V. @DataProviders
- VI. Types of execution
- VII. @Parameters
- VIII. Configure Annotations
- IX. Base class
- X. Advance Report
- XI. @Listeners

5. Framework

- I. Explanation
- II. Types of framework
- III. Stages of framework

6. Cucumber

7. GitHub

8. Maven

9. Jenkins

Chapter 0: Introduction

1. Difference between selenium 3 vs selenium 4

Feature / Difference	Selenium 3	Selenium 4
Driver Setup	Requires setProperty()	No need for setProperty()
Communication Protocol	JSON Wire Protocol	W3C Protocol
Explicit Wait Time Units	Only seconds	Seconds, minutes, hours, etc.
Relative Locators	Not available	Available
Opening New Window/Tab	newWindow() not available	newWindow() available
Minimize Browser Window	minimize() not available	minimize() available

2. Java project vs Maven Project

Aspect	Java Project	Maven Project
Dependency Management	Manual (add JARs to classpath)	Automatic (via pom.xml)
Build Automation	Manual (compile/run manually)	Automated via Maven lifecycle
Folder Structure	No fixed structure	Standard directory layout
External JAR Handling	Add manually	Add in pom.xml, Maven fetches
Best For	Simple or learning projects	Real-world/test automation projects

3. CRM introduction

- **CRM (Customer Relationship Management)** is a software system that helps companies manage interactions with customers.
- **B2B application**
 - They don't sell the product to customers
 - They build applications and other business people use that to run their internal business.
 - CRM application acts like a template that can be used by other business people to run their business
 - By referring CRM, TestYantra will build an application based on requirements given by Qspiders.

◆ Key Features of a CRM System:

- Contact Management – Stores customer data like name, email, phone, etc.
- Sales Tracking – Tracks leads, opportunities, and deals.
- Marketing Automation – Sends campaigns and tracks results.
- Customer Support – Manages service tickets and queries.
- Analytics & Reporting – Provides reports on sales, service, performance.

◆ Examples of CRM Tools:

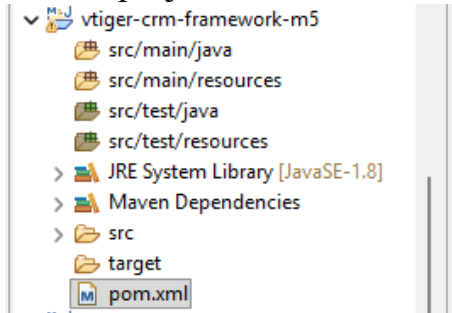
- a. Salesforce
- b. Vtiger CRM
- c. Zoho CRM
- d. HubSpot CRM
- e. Microsoft Dynamics 365

◆ Why CRM is Important:

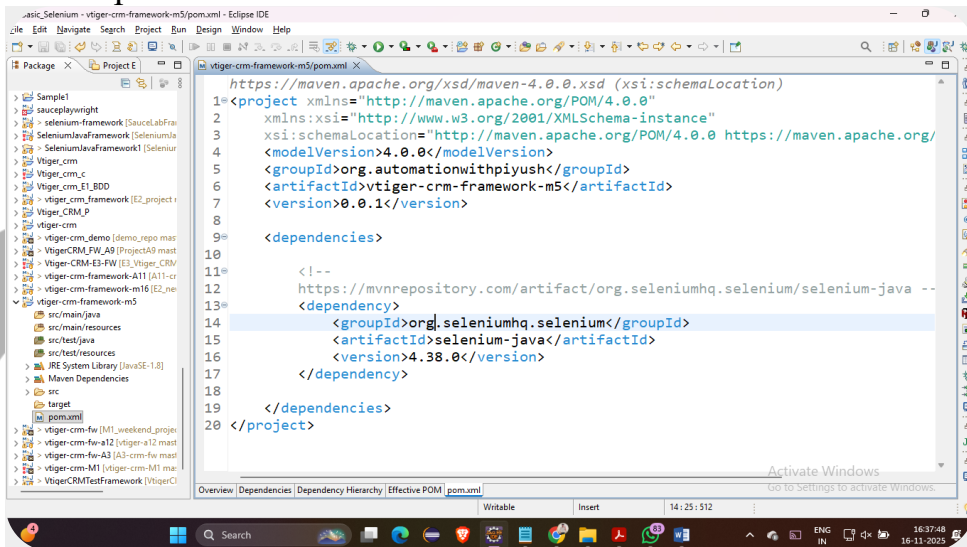
- Improves customer satisfaction
- Increases sales and productivity
- Helps in decision making using data
- Ensures better communication and follow-up

4. V-Tiger Test scripts

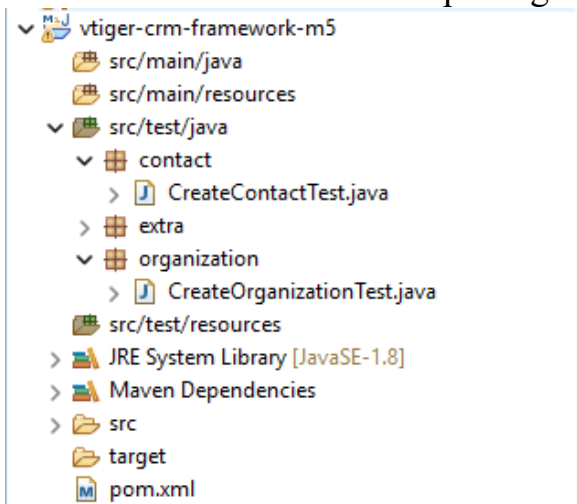
1. Maven project creation



2. Add dependencies



3. Create 2 classes inside 2 diff packages (contact & organization)



4. CreateContactTest.java

```
package contact;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;

public class CreateContactTest {
    public static void main(String[] args) throws InterruptedException {
        //      open the browser
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(15));

        //      Login
        driver.get("http://localhost:8888/");

        WebElement username = driver.findElement(By.name("user_name"));
        WebElement password = driver.findElement(By.name("user_password"));
        username.sendKeys("admin");
        password.sendKeys("password" + Keys.ENTER);

        //      Create contact
        driver.findElement(By.linkText("Contacts")).click();
        driver.findElement(By.cssSelector("img[alt='Create Contact...']")).click();

        //      Filling Form
        String lastName = "Kumar";
        WebElement lastNameField = driver.findElement(By.name("lastname"));
        lastNameField.sendKeys(lastName);

        //      Save
        driver.findElement(By.cssSelector("input[title='Save [Alt+S]']")).click();

        //      Verification
        String actLastName = driver.findElement(By.id("dtlview_Last Name")).getText();

        if (actLastName.equals(lastName)) {
            System.out.println("Contact Successfully created !!!");
        } else {
            System.out.println("Failed.....");
        }

        //      logout
    }
}
```



```

WebElement profile =
driver.findElement(By.cssSelector("img[src='themes/softed/images/user.PNG']"));

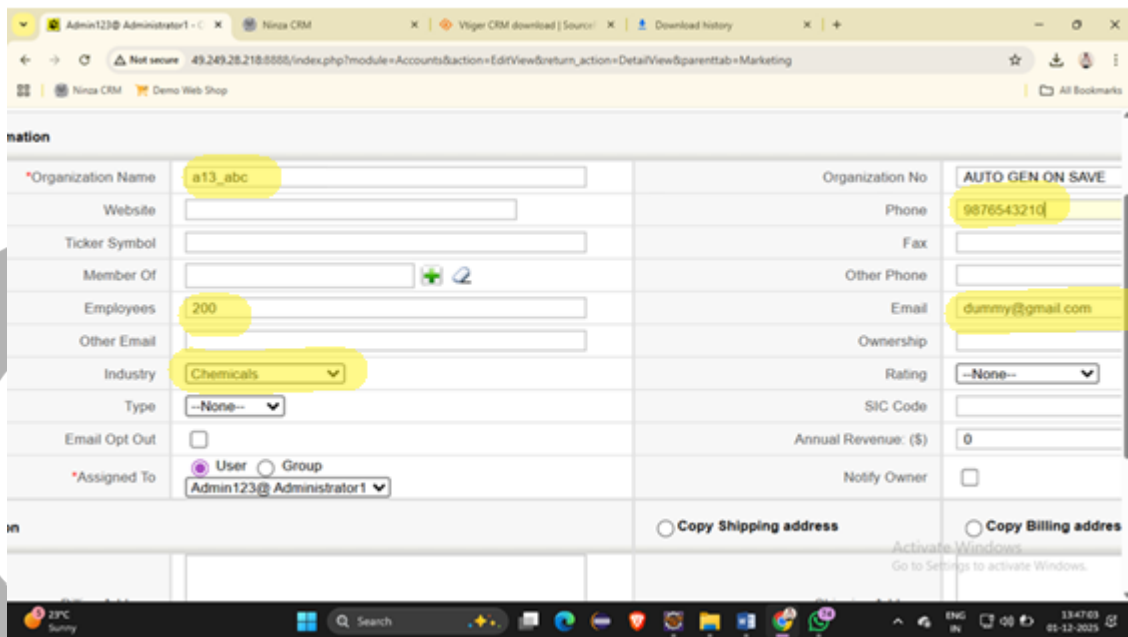
Actions act = new Actions(driver);
act.moveToElement(profile).build().perform();

driver.findElement(By.linkText("Sign Out")).click();

//          close the browser
Thread.sleep(10000);
driver.quit();
}
}

```

5. CreateOrganizationTest.java



```
package organization;
```

```
import java.time.Duration;
```

```

import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;

```

```
import extra.GenRanNum;
```

```

public class CreateOrganizationTest {
    public static void main(String[] args) throws InterruptedException {
//          open the browser
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(15));

//          Login

```

```

driver.get("http://localhost:8888/");

WebElement username = driver.findElement(By.name("user_name"));
WebElement password = driver.findElement(By.name("user_password"));
username.sendKeys("admin");
password.sendKeys("password" + Keys.ENTER);

// Create organization
driver.findElement(By.linkText("Organizations")).click();
driver.findElement(By.cssSelector("img[alt='Create Organization...']")).click();

// Filling Form
String orgName = "MakunaiGlobal_" + GenRanNum.generateRandomNumber();
WebElement orgField = driver.findElement(By.name("accountname"));
orgField.sendKeys(orgName);

// Save
driver.findElement(By.cssSelector("input[title='Save [Alt+S]']")).click();

// Verification
String actOrgName = driver.findElement(By.id("dtlview_Organization Name")).getText();

if (actOrgName.equals(orgName)) {
    System.out.println("Organization Succesfully created !!!");
} else {
    System.out.println("Failed.....");
}

// logout
WebElement profile =
driver.findElement(By.cssSelector("img[src='themes/softed/images/user.PNG']"));

Actions act = new Actions(driver);
act.moveToElement(profile).build().perform();

driver.findElement(By.linkText("Sign Out")).click();

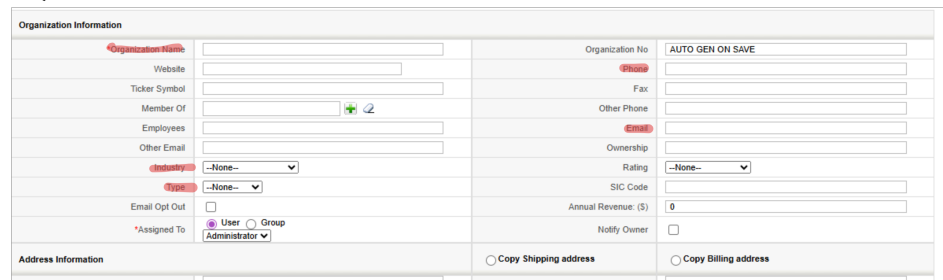
// close the browser
Thread.sleep(10000);
driver.quit();
}
}

```

CHAPTER 1 : DATA DRIVEN TESTING

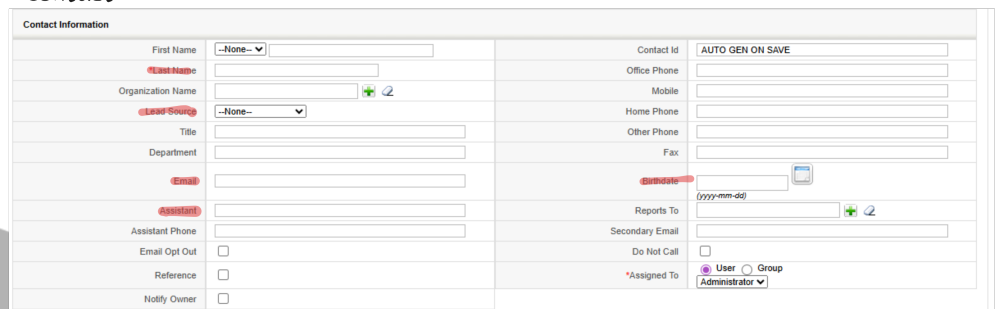
Organization

browser
url
username
password
orgName
industry
type
phone
email



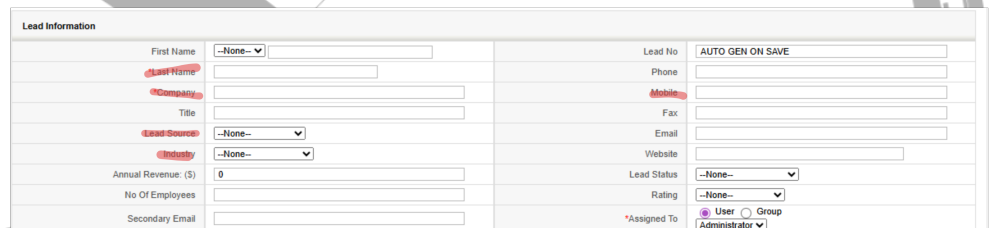
Contact

browser
url
username
password
lastName
leadSource
email
assistant
birthdate



Leads

browser
url
username
password
lastName
compName
leadsource
industry
mobile



Types of Test Data in Automation

Common Data

- The Data which is common for **all** the test scripts/applications/projects is called as common data
- Example: url, usernames, passwords
- Usually stored in a file like: CommonData.properties.

Test Script Data

- The Data which is common for **particular** test scripts/applications/projects is called as Test script data
- Example: product names, Organization names
- Usually stored in a file like: testScriptData.xlsx.

Why DDT?

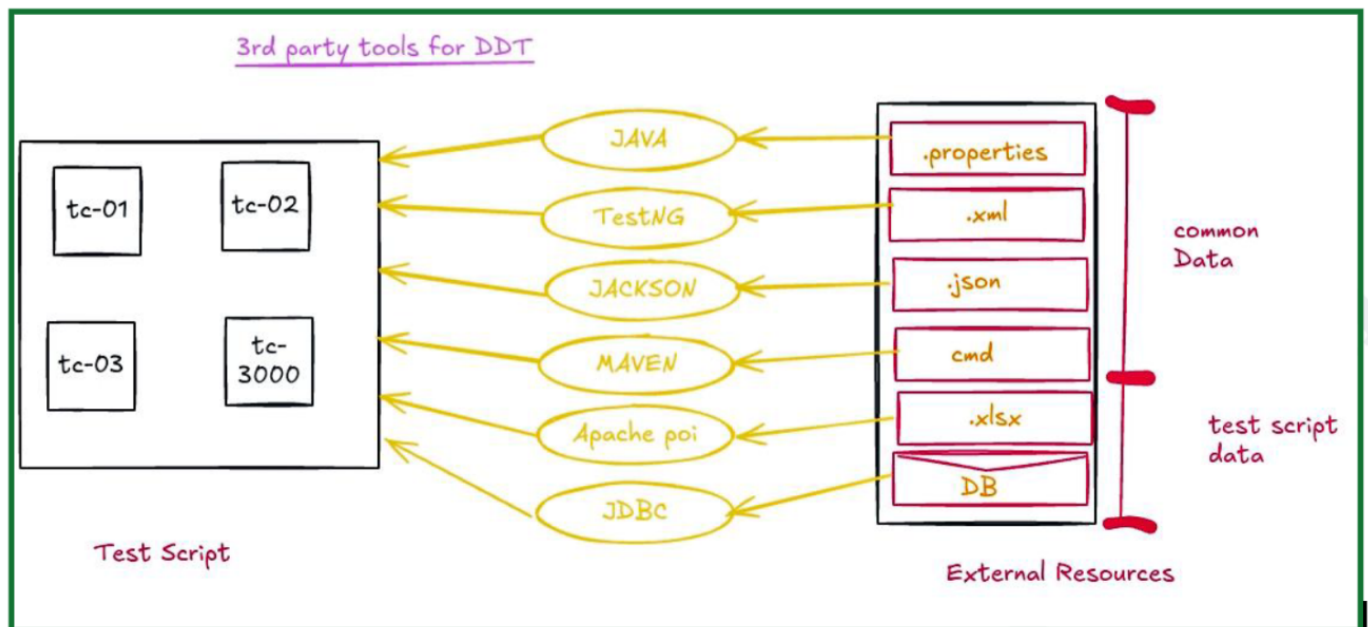
- The first rule of automation, we should never ever hardcode the test data into our scripts.
- Hardcoding test data makes maintenance and updates difficult.
- DDT helps separate test logic from test data, improving reusability and flexibility.

What is Data Driven Testing?

- Testing the application with the help of external resources and running the test scripts is called as DDT.

Advantages of DDT

1. Maintenance is easy
2. Modification is easy
3. Cross browser testing
4. Multi environment testing
5. Testing with multiple user credentials.
6. Re-running the same test with different inputs is easy.
7. Perfect for Agile methodology, where frequent builds require to run daily with different data sets.



Data Driven Testing Using Properties File

1. What is a Properties File?

- A Properties File is a Java-supported file used to store key-value pairs.
- Commonly used to store configuration or test data (common data).

1.1. What are the rules to create Properties File:

- #1. Data should be in key value format and in compulsorily String
- #2. key value should be separated with space or = or :
- #3. One pair per line
- #4. Keys should be UNIQUE
- #5. Extension must be .properties

url https://localhost:8888

bro chrome

un admin

pwd manager

2. Why do we Use a Properties File?

1. Lightweight and easy to maintain.
2. Faster to read than Excel or other formats.

Java provides a built-in **Properties class** for reading these files.

3. How to Read Data from a Properties File in Java?

```
// step 1> Create a java representation object of the physical file
// /vtiger-crm-framework-m5/src/test/resources/commondata.properties
// modified to
// ./src/test/resources/commondata.properties
// FileInputStream fis = new FileInputStream("./src/test/resources/commondata.properties");

// Properties class
// step 2> by using load(), load all the keys
// Properties pObj = new Properties();
// pObj.load(fis);

// step 3> by using getProperty("key"), get the value
// String value = pObj.getProperty("key");
// System.out.println(value);
```

★ Note:

Properties file is faster and more efficient for reading data compared to Excel.

The screenshot shows the Eclipse IDE with two open files. The left file, `*GetDataFromPropFile.java`, contains the following Java code:

```

1  import java.io.*;
2  import java.util.*;
3  import java.io.IOException;
4  import java.util.Properties;
5
6  class GetDataFromPropFile {
7
8      public static void main(String[] args) throws IOException {
9          // Step 1> create java representation object of the physical file
10         FileInputStream fis = new FileInputStream("./src/test/resources/commondata.properties");
11
12         // Step 2> load all the keys by using load(fis)
13         Properties pObj = new Properties();
14         pObj.load(fis);
15
16         // Step 3> get the value by passing key in getProperty(key)
17         String BROWSER = pObj.getProperty("bro");
18         System.out.println(BROWSER);
19
20         fis.close();
21     }
22 }

```

The right file, `commondata.properties`, contains the following key-value pairs:

```

1 bro chrome
2 url http://localhost:8888
3 un admin
4 pwd password

```

The console output at the bottom shows the execution of the program, displaying the value of the `bro` key:

```

<terminated> GetDataFromPropFile (4) [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe (29-May-2026, 12:12:07 pm - 12:12:08 pm elapsed: 0:00:00.340) [pid: 23344]
chrome
http://localhost:8888
admin
password

```

Feature	File	FileReader	FileInputStream
Represents file path	✓	✗	✗
Reads data	✗	✓	✓
Reads characters	✗	✓	✗
Reads bytes	✗	✗	✓
Good for text files	✗	✓	✓
Good for binary files	✗	✗	✓
Used in Selenium frameworks	Sometimes	Very common	Very common

Reading Data from JSON File

What is JSON?

JSON = JavaScript Object Notation

It is a lightweight format used to store and exchange data.

Example:

```
{
  "username": "admin",
  "password": "admin123"
}
```

Advantages

Benefit	Explanation
External Test Data	No hardcoding
Reusable	Same data for multiple tests
Easy Maintenance	Update JSON only
Framework Standard	Used in real projects
Supports Nested Data	Complex test scenarios

What is Simple JSON?

simple-json is a lightweight Java library used to:

- Read JSON
- Parse JSON
- Write JSON

Library Name:

json-simple

Maven Dependency

Add this in pom.xml

```
<dependency>
  <groupId>com.googlecode.json-simple</groupId>
  <artifactId>json-simple</artifactId>
  <version>1.1.1</version>
</dependency>
```

Create JSON File

File Name:

testData.json

Inside JSON File

```
{
  "username": "admin",
  "password": "password",
  "url": "https://localhost:8888"
}
```

Read JSON File in Java

Full Program

```
package jsonPractice;

import java.io.FileReader;

import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;

public class ReadJsonData {

    public static void main(String[] args) throws Exception {

        // Step 1: Create JSONParser object
        JSONParser parser = new JSONParser();

        // Step 2: Pass JSON file path
        FileReader reader = new FileReader(
            "./src/test/resources/testData.json");

        // Step 3: Parse JSON file
        Object obj = parser.parse(reader);

        // Step 4: Convert object into JSONObject
        JSONObject jsonObject = (JSONObject) obj;

        // Step 5: Read data using key
        String username = (String) jsonObject.get("username");
        String password = (String) jsonObject.get("password");
        String url = (String) jsonObject.get("url");

        // Step 6: Print data
        System.out.println(username);
        System.out.println(password);
        System.out.println(url);
    }
}
```

Output

```
admin
password
https://localhost:8888
```

Common Exceptions

FileNotFoundException

Cause:

Wrong file path

Fix:

Check JSON file location

ParseException

Cause:

Invalid JSON format

Example mistake:

✗ Wrong

```
{
  "username": "admin",
}
```

Trailing comma breaks JSON.

Interview Questions

Q1. Why JSON is used in Selenium?

Answer:

JSON is used for external test data management.
It improves framework maintainability and avoids hardcoding.

Q2. Difference between JSON and Excel?

JSON	Excel
Lightweight	Heavy
Faster	Slower
Better for APIs	Better for manual viewing
Structured	Tabular

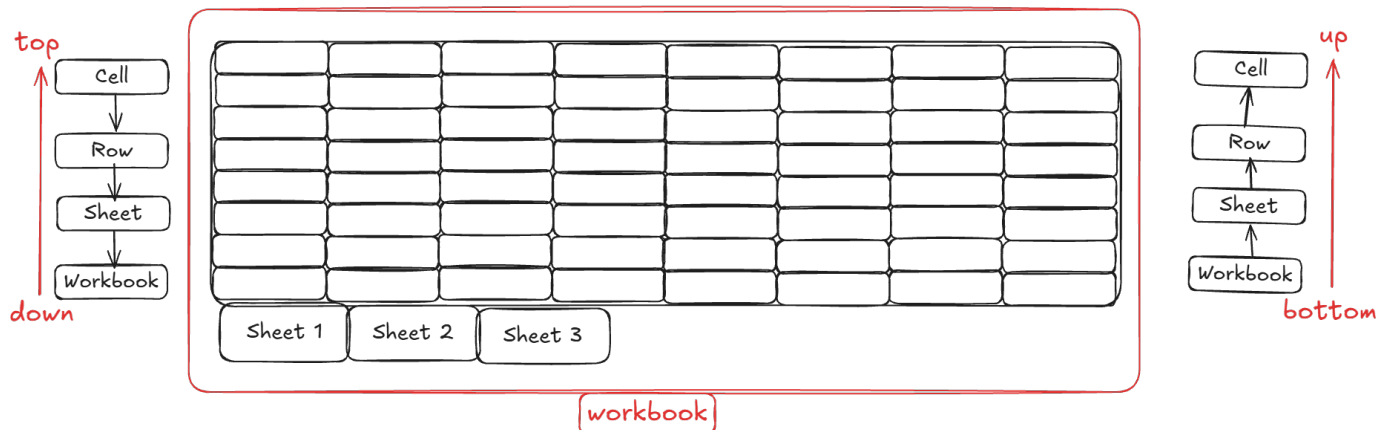
Q3. What is JSONObject?

Answer:

JSONObject is a class used to represent JSON data in key-value pair format.



Data Driven Testing Using Excel File (Apache POI)



Apache POI for Excel Handling

What is Apache POI?

- Apache POI is an open-source **Java library** that allows reading and writing of Microsoft Office formats.
- It supports formats like:
 - .xls and .xlsx (Excel)
 - .docx (Word)
 - .pptx (PowerPoint)
- In real-world automation projects, companies prefer to use **Excel** for test data due to its structured tabular format.

Apache POI Setup (Maven)

- Open your Maven Project in IDE.
- Open the pom.xml file.
- Visit: <https://mvnrepository.com>
- Search for: apache poi
- Add the following dependencies inside <dependencies> tag:

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi</artifactId>
  <version>5.5.1</version>
</dependency>
```

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.5.1</version>
</dependency>
```

Apache POI

WorkBookFactory <<C>>

create(fis)

[it will not create any thing,
it will open the workbook in read mode]

WorkBook <<I>>

getSheet("sheetname")

close()

write(fos)

Sheet <<I>>

getRow(rowNumber)

getLastRowNum();

Row <<I>>

getCell(cellNumber)

getLastCellValue()

Cell <<I>>

getStringCellValue()

getNumericCellValue()

getBooleanCellValue()

Get the data from excel file

```
//      step 1> get the java representation object of the physical file
//      /vtiger-crm-framework-m5/src/test/resources/testScriptData.xlsx
//      modified to
//      ./src/test/resources/testScriptData.xlsx
FileInputStream fis = new FileInputStream("./src/test/resources/testScriptData.xlsx");

//      step 2> get the access of workbook
Workbook wb = WorkbookFactory.create(fis);

//      step 3> get the access of sheet
Sheet sheet = wb.getSheet("dummy");

//      step 4> get the access of row
Row row = sheet.getRow(7);

//      step 5> get the access of cell
Cell cell = row.getCell(0);

//      step 6> get the data
String data = cell.getStringCellValue();
//      cell.getNumericCellValue();
//      cell.getBooleanCellValue();
System.out.println(data);

//      don't forget to close the workbook
wb.close();
```

Generic Utility

Utility/method/actions/library

Generic Utility

-> the methods which can be used for all the test script/application/project.

Business Utility

-> the methods which can be used for particular test script/application/project.

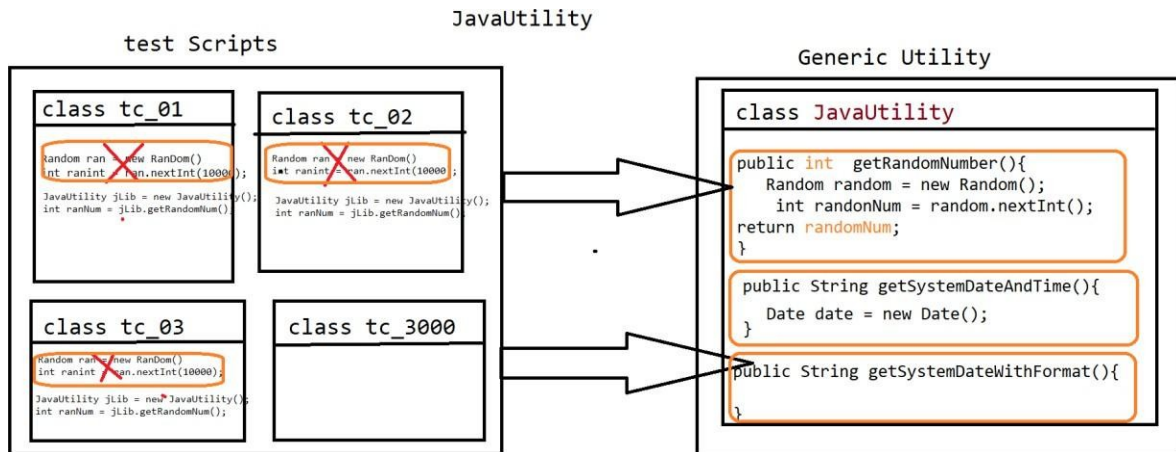
What is Generic components in Automation Framework?

- it's one of the automation framework components which is common for all the application
- its collection of generic class contains reusable methods / libraries
- The methods which can be used to any application is called Generic/common methods

What is the advantages of Generic components?

1. Reusability of code
2. Code Optimization
3. Test script development is faster
4. Test scripts Code readability
5. Generic libraries are common to all automation projects
6. Avoid duplicate Code
7. no need to remember the syntax every time, just create once & use multiple times

1. Java Utility Libraries

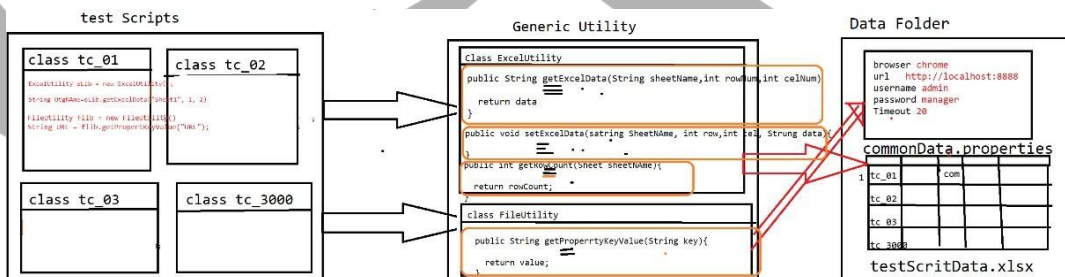


➔ Java Utility is one class in generic component, which contain java specific methods which can be used across the test Scripts / Application

➔ its contains several generic reusable methods like

- ⇒ `getRandomNum()` : it's used to generate random number for every invocation
- ⇒ `getSystemDate()` : it's used to generate system date and time

2. File Utility



- ⇒ As per the rule of automation, data should not be hardcoded with in the test scripts, so that to get the data from external file like Excel & .propertes file We go for ExcelUtility & FileUtility
- ⇒ Excel Utility class is developed using apache Poi libraries, which is used to read the data from Excel
- ⇒ FileUtility is used to get the data from .properties file

Version v1.0 - Raw file with hardcoded data

```
package crm.organization;

import java.io.IOException;
import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.interactions.Actions;

public class CreateOrgTest {
    public static void main(String[] args) throws InterruptedException,
    IOException {

//        open browser
        WebDriver driver = null;

        String browser = "chrome";

        if (browser.equals("chrome")) {
            driver = new ChromeDriver();
        } else if (browser.equals("edge")) {
            driver = new EdgeDriver();
        } else if (browser.equals("firefox")) {
            driver = new FirefoxDriver();
        } else {
            driver = new ChromeDriver();
        }

        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(15));

//        login
        driver.get("https://localhost:8888");

        WebElement un = driver.findElement(By.name("user_name"));
        un.sendKeys("admin");

        WebElement pwd = driver.findElement(By.name("user_password"));
        pwd.sendKeys("password");

        WebElement loginButton = driver.findElement(By.id("submitButton"));
        loginButton.click();

//        create organization
        driver.findElement(By.LinkText("Organizations")).click();
```



```

        driver.findElement(By.cssSelector("img[title='Create
Organization...']")).click();

//        fill the form
        String orgName = "qsp" + System.currentTimeMillis()/1000;
        WebElement orgField = driver.findElement(By.name("accountname"));

        orgField.sendKeys(orgName);

//        save
        driver.findElement(By.className("save")).click();

//        verification
        String actOrgName = driver.findElement(By.id("dtlview_Organization
Name")).getText();

        if (actOrgName.equals(orgName)) {
            System.out.println("org created successfullyyyy !!!");
        } else {
            System.out.println("Could not create organization");
        }

//        logout
        WebElement profile =
driver.findElement(By.cssSelector("img[src='themes/softed/images/user.PNG']"));
;

//        hover on profile
        Actions act = new Actions(driver);
        act.moveToElement(profile).build().perform();

//        click on sign out link
        driver.findElement(By.LinkText("Sign Out")).click();

//        close the browser
        Thread.sleep(3000);
        driver.quit();
    }
}

```

Version v2.0 - DDT

```
package crm.organization;

import java.io.FileInputStream;
import java.io.FileReader;
import java.io.IOException;
import java.time.Duration;

import org.apache.poi.ss.usermodel.Cell;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;
import org.apache.poi.ss.usermodel.WorkbookFactory;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import org.json.simple.parser.ParseException;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.interactions.Actions;

public class CreateOrgTest {
    public static void main(String[] args) throws InterruptedException,
    IOException, ParseException {

//        get the data from json file
        FileReader fr = new FileReader("./src/test/resources/commondata.json");
        JSONParser parser = new JSONParser();
        Object obj = parser.parse(fr);
        JSONObject jsonObj = (JSONObject) obj;
        String browser = jsonObj.get("bro").toString();
        String url = jsonObj.get("url").toString();
        String username = jsonObj.get("un").toString();
        String password = jsonObj.get("pwd").toString();

//        get the data from Excel file
        FileInputStream fis = new
FileInputStream("./src/test/resources/demo.xlsx");
        Workbook wb = WorkbookFactory.create(fis);
        Sheet sh = wb.getSheet("org");
        Row row = sh.getRow(2);
        Cell cell = row.getCell(0);
        String orgName = cell.getStringCellValue() +
System.currentTimeMillis()/1000;

//        open browser
        WebDriver driver = null;

        if (browser.equals("chrome")) {
            driver = new ChromeDriver();
```

```

    } else if (browser.equals("edge")) {
        driver = new EdgeDriver();
    } else if (browser.equals("firefox")) {
        driver = new FirefoxDriver();
    } else {
        driver = new ChromeDriver();
    }

    driver.manage().window().maximize();
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(15));

//    login
    driver.get(url);

    WebElement un = driver.findElement(By.name("user_name"));
    WebElement pwd = driver.findElement(By.name("user_password"));
    WebElement loginButton = driver.findElement(By.id("submitButton"));

    un.sendKeys(username);
    pwd.sendKeys(password);
    loginButton.click();

//    create organization
    driver.findElement(By.LinkText("Organizations")).click();
    driver.findElement(By.cssSelector("img[title='Create
Organization...']")).click();

//    fill the form
    WebElement orgField = driver.findElement(By.name("accountname"));

    orgField.sendKeys(orgName);

//    save
    driver.findElement(By.className("save")).click();

//    verification
    String actOrgName = driver.findElement(By.id("dtlview_Organization
Name")).getText();

    if (actOrgName.equals(orgName)) {
        System.out.println("org created successfullyyyy !!!");
    } else {
        System.out.println("Could not create organization");
    }

//    logout
    WebElement profile =
driver.findElement(By.cssSelector("img[src='themes/softed/images/user.PNG']"));

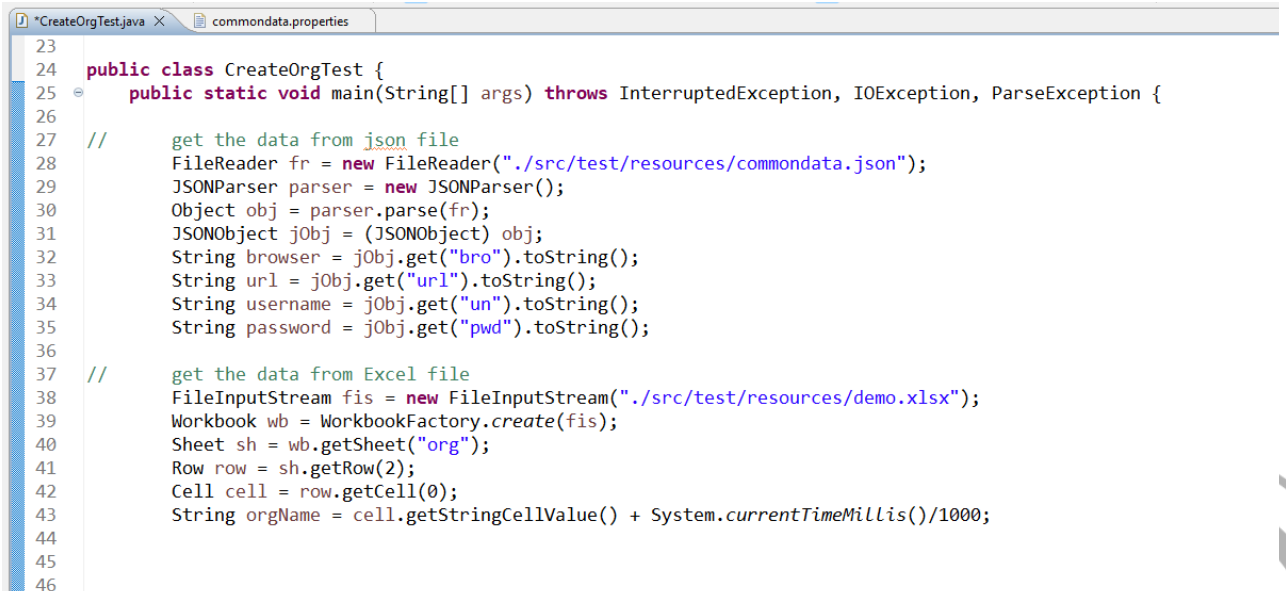
//    hover on profile
    Actions act = new Actions(driver);
    act.moveToElement(profile).build().perform();

//    click on sign out link
    driver.findElement(By.LinkText("Sign Out")).click();

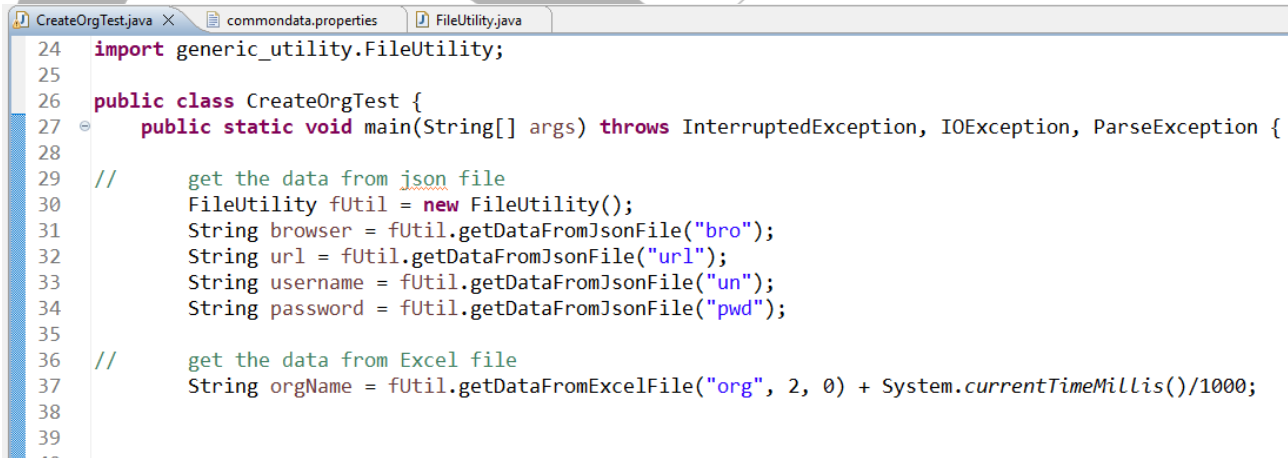
```

```
//      close the browser
      Thread.sleep(3000);
      driver.quit();
  }
}
```

Version v3.0 - Optimized Test scripts



```
23
24 public class CreateOrgTest {
25     public static void main(String[] args) throws InterruptedException, IOException, ParseException {
26
27         //      get the data from json file
28         FileReader fr = new FileReader("./src/test/resources/commondata.json");
29         JSONParser parser = new JSONParser();
30         Object obj = parser.parse(fr);
31         JSONObject jsonObj = (JSONObject) obj;
32         String browser = jsonObj.get("bro").toString();
33         String url = jsonObj.get("url").toString();
34         String username = jsonObj.get("un").toString();
35         String password = jsonObj.get("pwd").toString();
36
37         //      get the data from Excel file
38         FileInputStream fis = new FileInputStream("./src/test/resources/demo.xlsx");
39         Workbook wb = WorkbookFactory.create(fis);
40         Sheet sh = wb.getSheet("org");
41         Row row = sh.getRow(2);
42         Cell cell = row.getCell(0);
43         String orgName = cell.getStringCellValue() + System.currentTimeMillis()/1000;
44
45
46
```



```
24 import generic_utility.FileUtility;
25
26 public class CreateOrgTest {
27     public static void main(String[] args) throws InterruptedException, IOException, ParseException {
28
29         //      get the data from json file
30         FileUtility fUtil = new FileUtility();
31         String browser = fUtil.getDataFromJsonFile("bro");
32         String url = fUtil.getDataFromJsonFile("url");
33         String username = fUtil.getDataFromJsonFile("un");
34         String password = fUtil.getDataFromJsonFile("pwd");
35
36         //      get the data from Excel file
37         String orgName = fUtil.getDataFromExcelFile("org", 2, 0) + System.currentTimeMillis()/1000;
38
39
40
```

```
package crm.organization;
```

```
import java.io.FileInputStream;
import java.io.FileReader;
import java.io.IOException;
import java.time.Duration;
```

```
import org.apache.poi.ss.usermodel.Cell;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;
import org.apache.poi.ss.usermodel.WorkbookFactory;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import org.json.simple.parser.ParseException;
```

```

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.interactions.Actions;

import generic_utility.FileUtility;

public class CreateOrgTest {
    public static void main(String[] args) throws InterruptedException,
IOException, ParseException {

//        get the data from json file
        FileUtility fUtil = new FileUtility();
        String browser = fUtil.getDataFromJsonFile("bro");
        String url = fUtil.getDataFromJsonFile("url");
        String username = fUtil.getDataFromJsonFile("un");
        String password = fUtil.getDataFromJsonFile("pwd");

//        get the data from Excel file
        String orgName = fUtil.getDataFromExcelFile("org", 2, 0) +
System.currentTimeMillis() / 1000;

//        open browser
        WebDriver driver = null;

        if (browser.equals("chrome")) {
            driver = new ChromeDriver();
        } else if (browser.equals("edge")) {
            driver = new EdgeDriver();
        } else if (browser.equals("firefox")) {
            driver = new FirefoxDriver();
        } else {
            driver = new ChromeDriver();
        }

        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(15));

//        login
        driver.get(url);

        WebElement un = driver.findElement(By.name("user_name"));
        WebElement pwd = driver.findElement(By.name("user_password"));
        WebElement loginButton = driver.findElement(By.id("submitButton"));

        un.sendKeys(username);
        pwd.sendKeys(password);
        loginButton.click();

//        create organization
        driver.findElement(By.LinkText("Organizations")).click();
        driver.findElement(By.cssSelector("img[title='Create
Organization...']")).click();

```

```

//      fill the form
//      String orgName = "qsp" + System.currentTimeMillis()/1000;
//      WebElement orgField = driver.findElement(By.name("accountname"));

//      orgField.sendKeys(orgName);

//      save
//      driver.findElement(By.className("save")).click();

//      verification
//      String actOrgName = driver.findElement(By.id("dtlview_Organization
//      Name")).getText();

//      if (actOrgName.equals(orgName)) {
//          System.out.println("org created successfullyyyy !!!");
//      } else {
//          System.out.println("Could not create organization");
//      }

//      logout
//      WebElement profile =
//      driver.findElement(By.cssSelector("img[src='themes/softed/images/user.PNG']"));

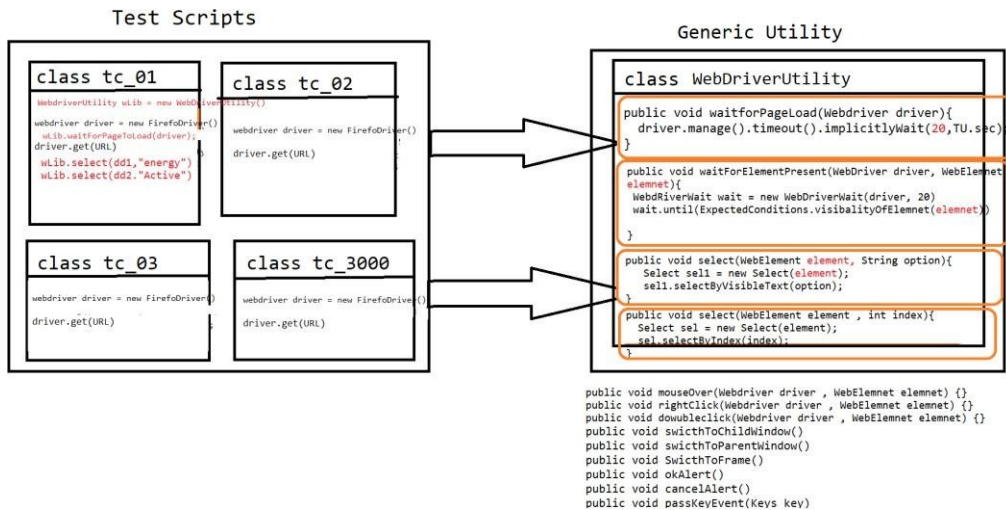
//      hover on profile
//      Actions act = new Actions(driver);
//      act.moveToElement(profile).build().perform();

//      click on sign out link
//      driver.findElement(By.LinkText("Sign Out")).click();

//      close the browser
//      Thread.sleep(3000);
//      driver.quit();
}
}

```

3. WebDriver Utility Libraries



PAGE OBJECT MODEL (POM)

What is POM?

POM (Page Object Model) is a design pattern used in Selenium Automation Frameworks to separate the web element locators from the test scripts.

Instead of directly writing locators inside test cases, all page elements are stored in separate Java classes called **Page Classes**.

Definition

Page Object Model (POM) is a design pattern that creates an Object Repository for web elements and keeps test scripts independent from UI locators.

Why Do We Need POM?

Without POM

```
driver.findElement(By.id("username")).sendKeys("admin");  
driver.findElement(By.id("password")).sendKeys("admin123");  
driver.findElement(By.id("loginBtn")).click();
```

Problems

- ✗ Hardcoded locators
- ✗ Difficult maintenance
- ✗ Code duplication
- ✗ Poor readability
- ✗ More effort when UI changes

With POM

```
LoginPage lp = new LoginPage(driver);
```

```
lp.enterUsername("admin");  
lp.enterPassword("admin123");  
lp.clickLogin();
```

Benefits

- ✓ Clean code
- ✓ Easy maintenance
- ✓ Reusable methods
- ✓ Better readability
- ✓ Less code duplication

Advantages of POM

1. Avoid hardcoding of locators
2. Maintenance of locators is easy
3. Modification of locators is easy
4. Reusability of the code
5. Readability of the code
6. We can handle StaleElementReferenceException.
7. AutoHealing is possible

Rules of POM

Rule 1

Number of Web Pages = Number of POM Classes

Web Page	POM Class
Login Page	LoginPage
Home Page	HomePage
Cart Page	CartPage

Rule 2

Number of Elements = Number of WebElement Variables

```
@FindBy(id="username")  
private WebElement username;
```

Rule 3

Declare WebElements as Private

```
private WebElement username;
```

Rule 4

Access Through Public Methods

```
public void enterUsername(String user)  
{  
    username.sendKeys(user);  
}
```

Rule 5

Initialize Using PageFactory

```
PageFactory.initElements(driver, this);
```

Steps to Create POM Class

Step 1

Create Page Class

```
public class LoginPage
{
}
```

Step 2

Store Web Elements

```
@FindBy(id="username")
private WebElement username;

@FindBy(id="password")
private WebElement password;

@FindBy(id="loginBtn")
private WebElement loginBtn;
```

Step 3

Initialize Elements

```
PageFactory.initElements(driver, this);
```

Step 4

Create Constructor

```
public LoginPage(WebDriver driver)
{
    PageFactory.initElements(driver, this);
}
```

Step 5

Create Business Methods

```
public void enterUsername(String user)
{
    username.sendKeys(user);
}

public void enterPassword(String pass)
{
    password.sendKeys(pass);
}

public void clickLogin()
{
    loginBtn.click();
}
```

Complete LoginPage Example

```
package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
```

```
public class LoginPage
{
    @FindBy(id="username")
    private WebElement username;

    @FindBy(id="password")
    private WebElement password;

    @FindBy(id="loginBtn")
    private WebElement loginBtn;

    public LoginPage(WebDriver driver)
    {
        PageFactory.initElements(driver, this);
    }

    public void enterUsername(String user)
    {
        username.sendKeys(user);
    }

    public void enterPassword(String pass)
    {
        password.sendKeys(pass);
    }

    public void clickLogin()
    {
        loginBtn.click();
    }

    public void login(String user,String pass)
    {
        username.sendKeys(user);
        password.sendKeys(pass);
        loginBtn.click();
    }
}
```

What is PageFactory?

Definition

PageFactory is a Selenium class used to initialize web elements declared using `@FindBy` annotation.

Without PageFactory:

```
@FindBy(id="username")
WebElement username;
```

will remain NULL.

When accessed:

```
username.sendKeys("admin");
```

It throws:

NullPointerException

Syntax

```
PageFactory.initElements(driver, this);
```

Parameters

1. Driver Reference

driver

2. Current Class Object

this

Common Annotations in PageFactory

@FindBy

Locate single element.

```
@FindBy(id="username")
private WebElement username;
```

@FindBys

AND condition

```
@FindBys({
    @FindBy(tagName="input"),
    @FindBy(id="username")
})
private WebElement username;
```

@FindAll

OR condition

```
@FindAll({
@FindBy(id="username"),
@FindBy(name="username")
})
private WebElement username;
```

Difference Between POM and PageFactory

POM	PageFactory
Design Pattern	Selenium Class
Creates Page Objects	Initializes Elements
Organizes Framework	Supports POM
Improves Maintainability	Removes NullPointerException
Independent Concept	Part of POM

Interview Answer

POM is a design pattern used to separate web elements and business logic from test scripts. PageFactory is a Selenium class used inside POM to initialize the web elements created using @FindBy annotations. We can say PageFactory is a subset of POM.

Drawback of @FindBy

Dynamic XPath Not Supported

Cannot do:

```
@FindBy(xpath="//td[text()=''+name+'"]")
```

Because values change dynamically.

Solution

Use:

```
driver.findElement(
By.xpath("//td[text()=''+name+'"]")
);
```

or

```
driver.findElements()
```

Interview Questions

Q1. What is POM?

A design pattern that stores web elements in separate page classes and keeps test scripts independent of locators.

Q2. Why is POM used?

To improve maintainability, readability, reusability, and reduce code duplication.

Q3. What is PageFactory?

A Selenium class used to initialize elements declared using @FindBy annotations.

Q4. Why use PageFactory.initElements()?

To initialize WebElements and avoid NullPointerException.

Q5. Difference between POM and PageFactory?

POM is a framework design pattern; PageFactory is a Selenium utility class used within POM.

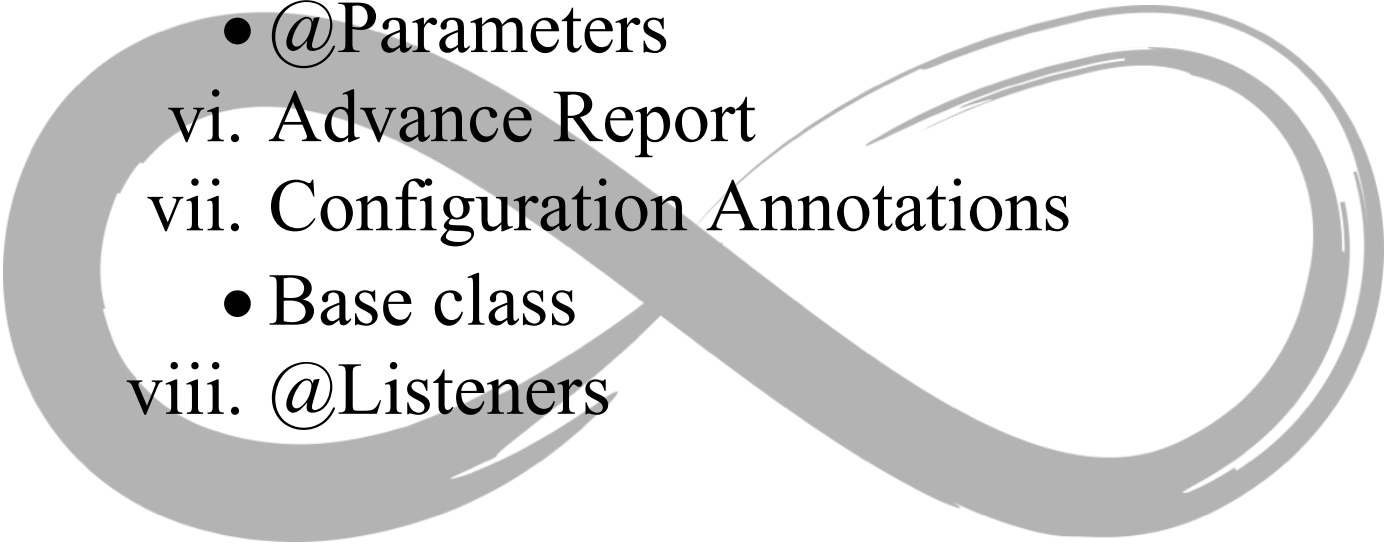
Q6. What are business methods?

Methods that perform user actions.

Example:

```
login();  
logout();  
addToCart();  
createCustomer();
```

TESTNG

- i. Introduction
 - ii. Basic Annotation
 - iii. Assertion
 - iv. Helper attributes
 - `@DataProvider`
 - v. Types of execution
 - `@Parameters`
 - vi. Advance Report
 - vii. Configuration Annotations
 - Base class
 - viii. `@Listeners`
- 

i. Introduction

1. TestNG is an open source **unit testing tool**
2. Here, NG stands for Next Generation
3. TestNG comes under TDD framework.
4. Unit testing via
 - TestNG Java, .NET
 - JUnit Java
 - NUnit .NET
 - PyDev Python
 - RSpec Ruby
 - Jasmine JavaScript
5. TestNG supports java and .net.
6. TestNG was developed as an **additional plugin** for Eclipse
7. In case of automation, TestNG will be used to develop all the scripts using TestNG **annotations**.
8. TestNG will be used to handle all framework component.
9. TestNG **.xml** is main controller of the selenium framework, where we start the execution.
10. TestNG is used by automation engineers because of its features and advantages

Installation of TestNG Plugin

1. Open eclipse and click on “help” major tab
2. Select “Eclipse Marketplace”.
3. Search “TestNG” in eclipse marketplace dialogue box
4. Click on “install” button under “TestNG for Eclipse”
5. Click on “confirm”
6. Click on “I agree to license agreement” radio button and then “next”
7. Select “install anyway” and then finish
8. **Click on select all, trust selected (this box will come twice)**
9. Click on “restart now”

Why TestNG over JUnit?

TestNG is a more advanced and flexible testing framework compared to JUnit. It provides additional features that make automation testing more powerful and manageable, especially for large-scale projects.

1. More Annotations

- TestNG provides a richer set of annotations compared to JUnit, allowing greater control over test execution flow.
(e.g., `@BeforeSuite`, `@AfterSuite`, `@DataProvider`, `@Factory`, etc.)

2. Batch Execution & Parallel Execution

- Multiple test classes or suites can be executed together.
- Supports parallel execution to reduce total test execution time.

3. Assertions

- Built-in assertion mechanisms for verifying expected results effectively.

4. Enhanced Functionality

- **HTML Reports:** Automatically generates detailed HTML reports after execution.
- **Parallel Execution:** Runs multiple tests simultaneously for faster feedback.
- **Grouping Execution:** Tests can be grouped logically (e.g., Smoke, Regression, Sanity).
- **Additional Annotations:** More control over test configuration and dependency handling.
- **Simplified Batch Execution:** Easy to configure test batches through the `testng.xml` file.
- **ITestListeners:** Used for event tracking and custom actions such as capturing screenshots on test failure.
- **Retry Analyzer:** Automatically re-runs failed test scripts to handle flaky tests.

Advantage:

1. **Generate Reports**
2. **Batch Execution of Test Cases**
3. **Group Execution of Test Cases**
4. **Parallel execution of Test Cases**
5. **Perform Assertion**
6. **Prioritize the test cases**
7. **Add dependency of test cases**
8. **Can disable the test cases.**
9. **Add preconditions and post condition**
10. **Provides different annotations**

Rules to follow while using TestNG

1. Use `@Test` method instead of main method
2. Use `Reporter.log()` instead of printing statement

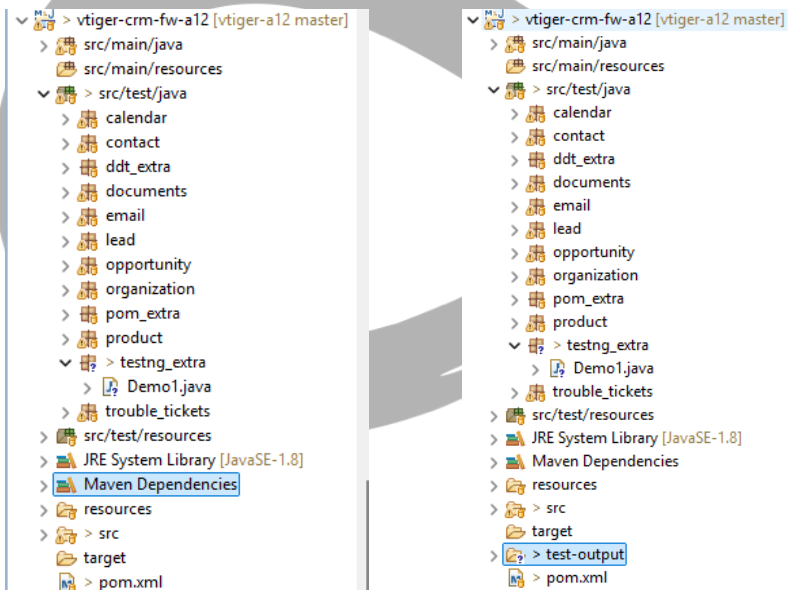
Reporting – Basic reports

- Basic reports are available in testNG to log the reports into HTML reports
- Reporter class is being used to generate the logs reports
- Test-output -> emailable-report.html (Open with web browser)
- Whenever we want all the details like how and why the test script got failed, which line is having which exception then we will go for Reporter class.
- Replace `System.out.println()` with `Reporter.log()`;

To Present in console as well as Remains in Reports

- `Reporter.log("Step – 1", true);`
- `true` will send the output in console also,
by default it is false (Which doesn't send to console)

Before execution vs After execution



i. Annotations:-

What is annotation?

- Annotation is the java template which is used to give information or metadata to JVM
- It can be used on top of variable or methods or classes.
- It always starts with “@” symbol

Types of Ann

1. Basic/Core

@Test

2. Configure Ann

@BeforeSuite

@BeforeTest

@BeforeClass

@BeforeMethod

@AfterMethod

@AfterClass

@AfterTest

@AfterSuite

3. Parameter Ann

@DataProvider

@Parameters

@Listeners

@Test

1. It is a core/basic annotation of TestNG
2. @Test annotation method **acts like a main method** in TestNG class.
3. Whenever we execute TestNG class, JVM always looks for @Test Annotation method to start the execution.
4. Without @Test Annotation method, TestNG class will not be executed.
5. **return type** should be “void”, **access specifier** should be public, but **method name** can be anything.
6. As per the Rule of TestNG, class Name should end with “**Test**” suffix.
(e.g., OrgTest, ContactTest...)
7. In one class we can keep multiple @Test annotation methods, but in real time we are going to maintain **maximum 10 to 15** @test annotation methods, because maintenance will be easy.

ii. Assertion

Introduction

- It is a feature provided by TestNG.
- It is used to **validate test scripts** and used to **verify expected results**
- As per the rule of automation every expected result should be verified with statement, because java “if – else” statements doesn’t have capability to fail the TestNG test script

```
If(true){  
    Sop(“first thing”);  
}  
else{  
    Sop(“another thing”);  
}
```

- Either one or another, There’s nothing called Failure
- Java if else block can’t say, Test case is getting failed
- So that, in order to verify the test case failed or not, we go for Assertion.
- There are 2 types of Assertions
 1. Hard Assert
 2. Soft Assert
- If any test case is getting failed, assertion will fail the TestNG test script by itself.

Difference between Hard and Soft Assert

Hard assert -> Assert <<C>>	Soft assert -> SoftAssert <<C>>
to verify mandatory fields we go for hard assert -> email or username in Sign up	To verify non-mandatory fields we go for soft assert -> Middle name in Sign Up
All methods are static in nature	All methods are non-static in nature
Test execution will be aborted if the condition is not met	Test execution will be continued till the end even if the assert condition is not met.
Does not need to invoke any method to capture the assertion Exception -> java.lang.AssertionError	To view assertions result, at the end of the test case, we need to invoke assertAll() . Exception -> java.lang.AssertionError
Whole test case gets failed if at least 1 hard assert fails	assertAll() extra line of code is required to track the failure status

Assertion

Hard Assert

-> mandatory fields

Soft Assert

-> optional fields

prefix
firstname*
middlename
lastname*
email*
country code
phone*
address
dob*
gender
country*
state
pincode
city
landmark
religion
password*
conf password*
category

Assert <<C>>

static methods

→ assertTrue(true)
→ assertFalse(false)
→ assertEquals("abc", "abc")
→ assertNotEquals("abc", "xyz")
→ assertNull(null)
→ assertNotNull(new Object())

SoftAssert <<C>>

non-static methods

→ assertTrue(true)
→ assertFalse(false)
→ assertEquals("abc", "abc")
→ assertNotEquals("abc", "xyz")
→ assertNull(null)
→ assertNotNull(new Object())
→ assertAll()

iii. Helper Attributes

1. Priority
2. DependsOnMethods
3. Enabled
4. AlwaysRun
5. InvocationCount
6. DataProvider
7. ThreadPoolSize

Priority

1. To change the “Order of Execution” we go for priority.
2. We can give positive, negative and 0.
3. lesser the value, earlier the execution.
4. If Priority is not specified, then the default value is 0.
5. Disadvantage:
 - a. Illogical / senseless flow

dependsOnMethods

1. it is used to create dependency between 2 test scripts
“test2” depends on “test1”
2. If “Test1” test script got passed,
“Test2” execution will be continued.
3. If “Test1” test script got failed,
“Test2” execution will be skipped.
4. The scope of dependsOnMethods is within particular TestNG Class.

Eg:-

```
@Test
public void test1() {
    Reporter.log("Account created Successfully");
}
```

```
@Test(dependsOnMethods= "test1")
public void test2(){
    Reporter.log("Account deleted Successfully");
}
```

5. Syntax:
@Test(dependsOnMethods = “method Name”)
6. Disadvantages:

As per the rule of automation we should not create dependency between test cases.

invocationCount

1. Executing the same test script multiple times, with the same set of data

2. Default value is 1.
3. Disadvantage:
 - a. We are executing the test scripts multiple time but with same data, what if we want different sets of data.

ThreadPoolSize

1. It is used to execute same test case multiple times with same test data **simultaneously**.
In order to use this, invocation count is mandatory.
2. By default, threadPoolSize is 0.
Eg:- @Test(invocationCount=10, threadPoolSize= 5)

Enabled

1. It is used to skip the test cases.
2. We can give Boolean values(True/False).
3. Default value is **true**.
4. If you want to skip some test cases, then you need to explicitly specify “false”.
Eg:- @Test(enabled= false)

DataProvider

1. Executing the same test script multiple times, with the different set of test data
2. It always returns 2-D object array, because we can pass any type of datatype
3. It helps us to execute same test multiple time with the different set of data, each test script should have dedicated @DataProvider annotation
4. It plays major role in “data driven framework”, where we need to test the application with huge amount of data like ecommerce and booking application

iv. Types of Execution:

Batch Execution

1. Collection of multiple test scripts together is called as a batch, To execute multiple test scripts through the xml file on a single click without any manual intervention is called batch execution.
2. In order to achieve batch execution, we go for TestNG.xml configuration file
3. TestNG xml file always start with **suite tag**
4. In one xml file we can invoke N-number of TestNG classes, but all the classes should be present within a project.

How to create TestNG xml file automatically through eclipse?

1. Select all the TestNG classes or packages -> right click-> select TestNG and click on "Convert to TestNG" -> click on finish.
2. Automatically you will get the wizard to create xml file
3. Rename the file with your chosen name
4. Click on finish

Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)
Suite					
SmokeTest	3	0	0	0	19,564

Class	Method	Start	Time (ms)
Suite			
SmokeTest — passed			
types_of_execution.Demo1	case1	1760168178027	6950
types_of_execution.Demo2	case2	1760168184986	6292
types_of_execution.Demo3	case3	1760168191282	6261

Parallel Execution:

Distributed Parallel execution:

- Distribute the test case across the multiple test runner & execute each <test>/Test runner in parallel is called Distributed Parallel execution
- we reduce the suite execution time, so that we can get the result early

- in order to achieve parallel execution we should enable parallel="tests" , then create multiple test runner & distribute the test-cases

```
<suite parallel="tests" name="Suite">
  <test parallel="tests" name="TestThread1">
    <classes>
      <class name="types_of_execution.Demo1" >
        <methods>
          <include name="case1"></include>
        </methods>
      </class>
    </classes>
  </test>

  <test parallel="tests" name="TestThread2">
    <classes>
      <class name="types_of_execution.Demo2" />
    </classes>
  </test>

  <test parallel="tests" name="TestThread3">
    <classes>
      <class name="types_of_execution.Demo3" />
    </classes>
  </test>
</suite>
```

Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)	In
Suite						
TestThread2	1	0	0	0	8,022	
TestThread1	1	0	0	0	8,062	
TestThread3	1	0	0	0	8,045	
Total	3	0	0	0	24,129	

Class	Method	Start	Time (ms)
Suite			
TestThread2 — passed			
types_of_execution.Demo2	case2	1760168723361	7965
TestThread1 — passed			
types_of_execution.Demo1	case1	1760168723362	8010
TestThread3 — passed			
types_of_execution.Demo3	case3	1760168723362	7993

-
- Thread count should be same as number of <test> testRunner

<parameter > is used to specify the browser data for each <test>

EG :

```
<suite name="Suite" parallel="tests" thread-count="2">
  <test name="Test-Runner-Chrome">
```

```
    <parameter name="BROWSER" value="chrome"/>
```

```
    <classes>
```

```

        <class
        name="com.vtiger.comcast.organizationtest.CreateOrganization"/>

        <class
        name="com.vtiger.comcast.organizationtest.SearchOrgTest"/>

    </classes>

</test> <!-- Test -->

<test name="Test-Runner-Firefox">

    <parameter name="BROWSER" value="firefox"/>

    <classes>

        <class name="com.vtiger.comcast.organizationtest.CreateOrganization"/>

        <class
        name="com.vtiger.comcast.organizationtest.SearchOrgTest"/>

    </classes>

</test>

</suite>

```

@Parameters annotation will be used to receive the data (Eg : browser , url, un, pwd etc.) from the XML file to test Scripts

Parallel Execution

Distributed parallel execution

- Distribute the test cases across the multiple test runner and execute each <test> test runner in parallel is called as Distributed parallel execution
- We reduce the suite execution time, so that we can get the result early
- In order to achieve parallel execution, we should enable parallel = "test" in <suite>, Then create multiple test runners and distribute the test cases

```

<suite parallel="tests" name="Suite">
    <test parallel="tests" name="Test1">
        <classes>
            <class name="package.Demo1" />
        </classes>
    </test>

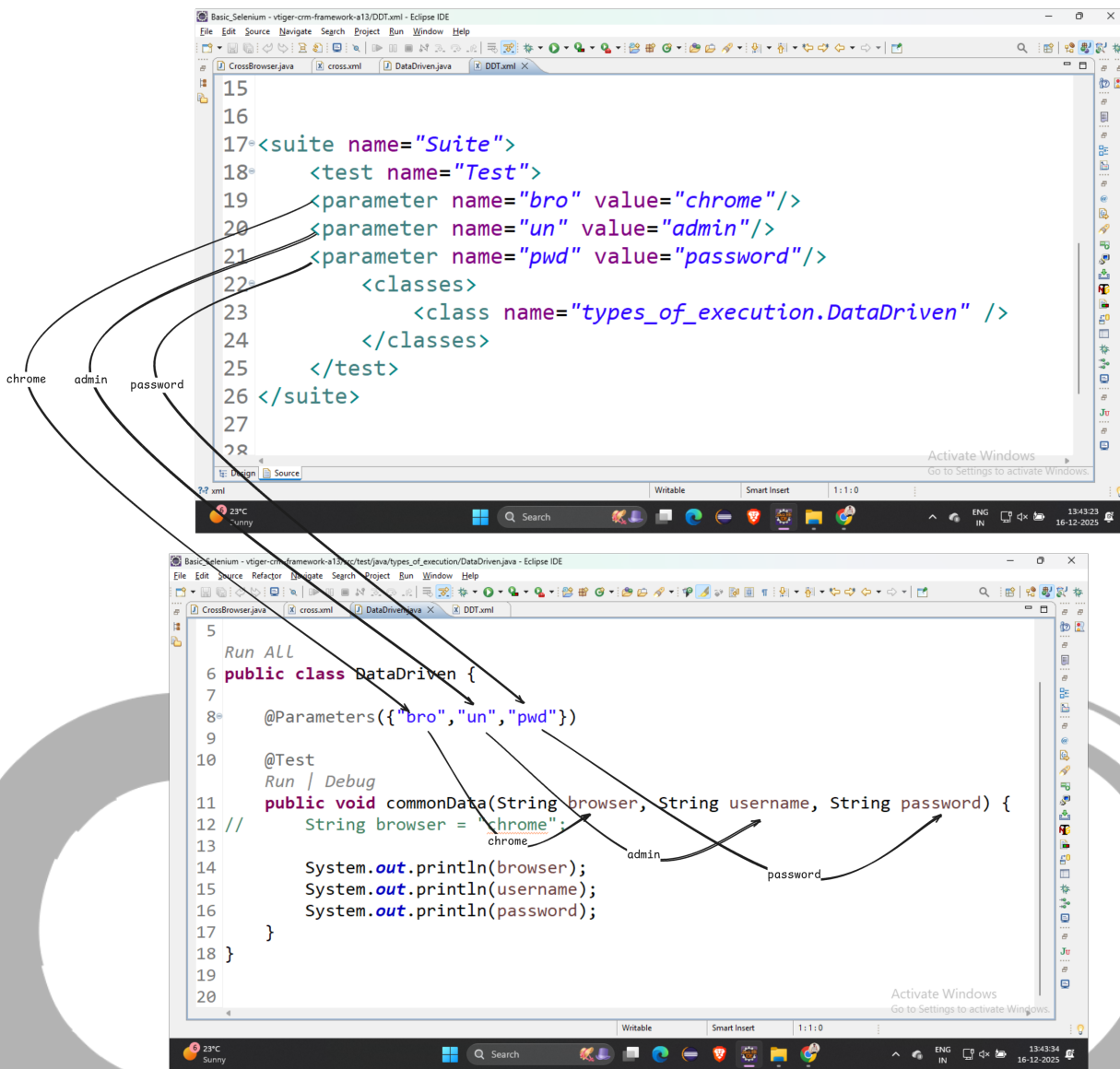
```

```
<test parallel="tests" name="Test2">
  <classes>
    <class name="package.Demo2" />
  </classes>
</test>
</suite>
```

Cross browser parallel execution / Browser Compatibility testing

- i. Execute same set of testcase in **different Browser** parallel is called **cross browser testing**
- ii. To achieve cross browser testing we should have <parameter> in XML file & @parameters annotation inside the testScript
- iii. We can pass parameters to testing test methods in 2 ways:
 - i. @DataProvider
 - ii. @Parameters





Group Execution:

- Collection of **similar** test scripts across the TestNG classes is called grouping Execution
- In order to achieve grouping execution, each & every test script should have group name, group name will be written along with annotation
- In grouping execution, all configure annotation should have group name, otherwise those annotation will not participate in grouping execution like @BeforeSuite @BeforeClass, @BeforeMethod etc.

EG :

```
@BeforeSuite(groups = {"smokeTest","regressionTest"})
```

```
public void configBS() {
```

```
    System.out.println("=====Execute BeforeSuite=====");
```

```
}
```

Smoke Test:

```
@Test(groups={"smokeTest"})
```

```
public void createCustomerTest(){
```

```
    System.out.println("execute createCustomerTest"); }
```

```
@Test(groups={"regressionTest"})
```

```
public void modifyCustomerTest(){
```

```
    System.out.println("execute modifyCustomerTest"); }
```

In order to invoke grouping execution, it should declare Group Key in testng.xml file & group keep should be declared before <test>, after <suite> tag

Smoke Test:

```
<suite name="Suite">
    <groups>
        <run>
            <include name="smokeTest"/>
        </run>
    </groups>
    <test name="Test">
        <classes>
            <class name="pac1.ProjectAndCustomerTest"/>
            <class name="pac2.ReportTest"/>
        </classes>
    </test>
</suite>
```

We can invoke multiple group-key in one XML File

```
<suite name="Suite">
    <groups>
        <run>
            <include name="smokeTest"/>
            <include name="regressionTest"/> </run>
        </groups>
    <test name="Test">
```

```
<classes>
  <class name="pac1.ProjectAndCustomerTest"/>
  <class name="pac2.ReportTest"/>
</classes>
</test>
</suite>
```



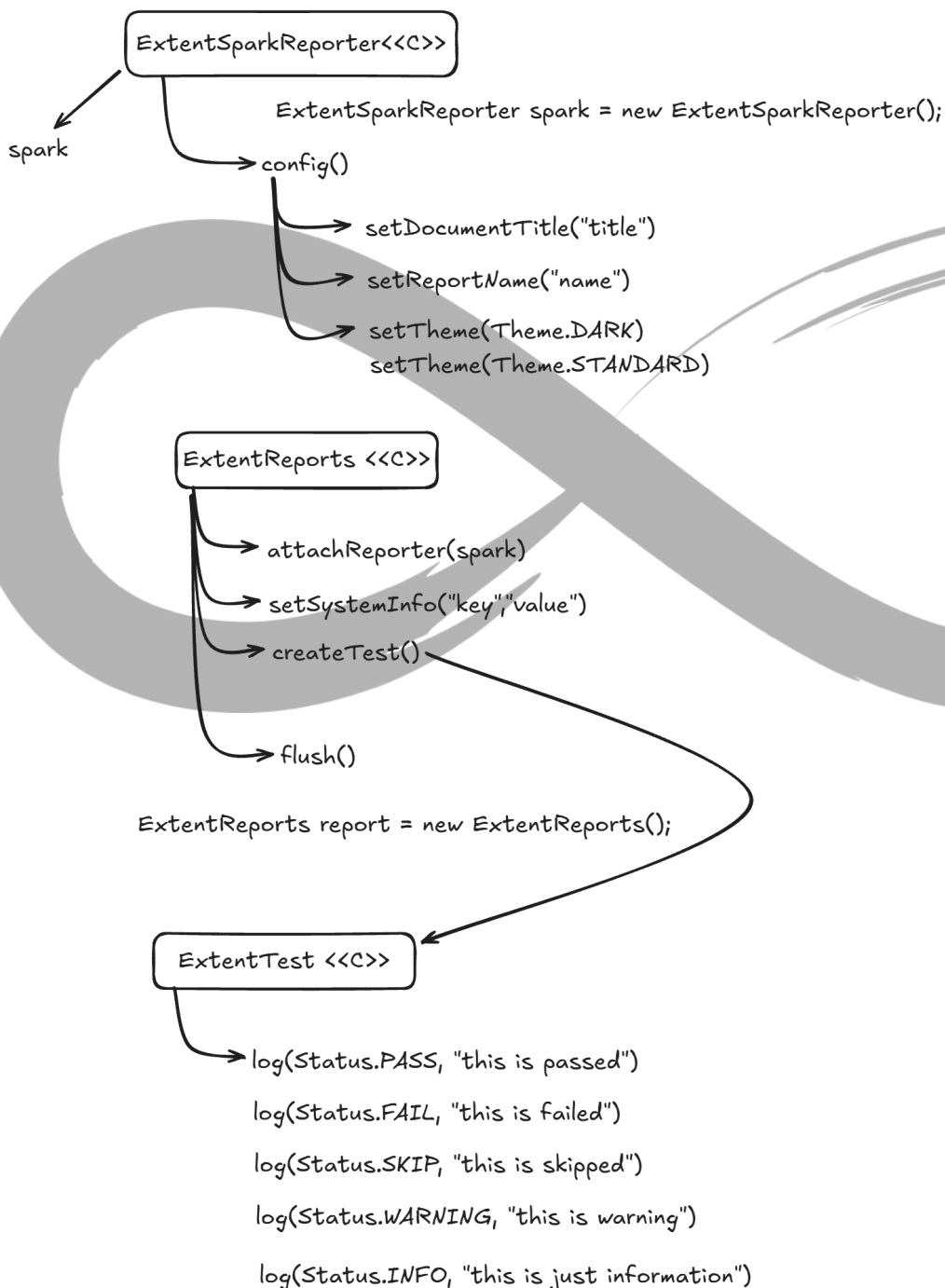
v. Advance Reports:

Extent Reports

- It provides advance features which helps in analyzing the test execution effectively

Advantages:

- It provides both high and low level reports
- It is an open source
- It provides eye-catching and lucid reports
- It has good documentation
- It can be easily integrated with CI/CD tools (jenkins)

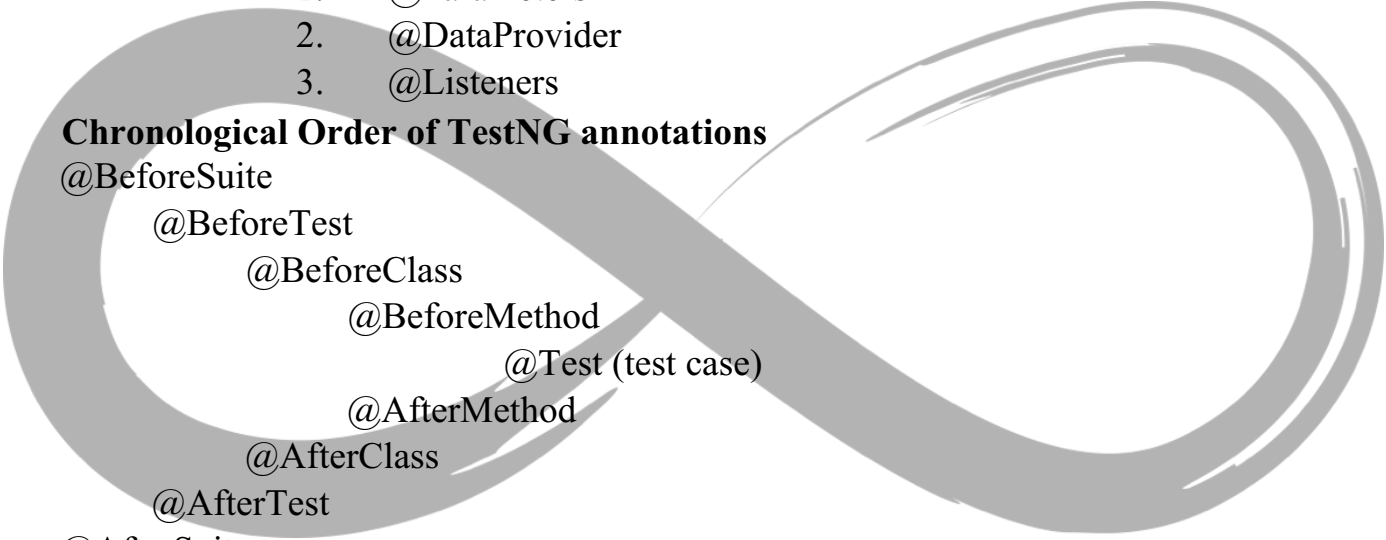


vi. Configuration Annotations

Types of Annotations

- A. Basic annotation
 - 1. `@Test`
- B. Configure annotation
 - 1. `@BeforeSuite`
 - 2. `@BeforeTest`
 - 3. `@BeforeClass`
 - 4. `@BeforeMethod`
 - 5. `@AfterMethod`
 - 6. `@AfterClass`
 - 7. `@AfterTest`
 - 8. `@AfterSuite`
- C. Parameter annotation
 - 1. `@Parameters`
 - 2. `@DataProvider`
 - 3. `@Listeners`

Chronological Order of TestNG annotations



`@BeforeSuite`
 `@BeforeTest`
 `@BeforeClass`
 `@BeforeMethod`
 `@Test` (test case)
 `@AfterMethod`
 `@AfterClass`
 `@AfterTest`
`@AfterSuite`

NOTE: As per the Rule of the Automation, there should not be any dependency between two test cases, every test case should be unique (it means every test should have fresh login & logout code).

@BeforeMethod and @AfterMethod

- 1. Before method annotations will be executed, before executing each `@Test` method in a class
- 2. After method annotation will be executed, after executing each `@Test` in a class
- 3. BeforeMethod & AfterMethod will not be executed, without `@test` annotation method, because they are configuration method

4. In order to implement similar pre- condition for all the test cases like “LOGIN code” we go for @ BeforeMethod
5. In order to implement similar post-condition for all the test cases like “LOGOUT code” we go for @ AfterMethod

@BeforeClass & @AfterClass

1. BeforeClass Annotation method will be executed, before Executing first @test in a class
2. AfterClass Annotation method will be executed, after executing all/last test-case within a class
3. BeforeClass & AfterClass annotations will be executed only once in an entire class execution.
4. It will be used to develop global configuration like launch browser, object initialization



vii. Listeners

Introduction

1. It is the feature available in TestNG, which helps to capture runtime events.
2. It is TestNG annotation, that literally ‘listens’ to the event in script
3. Listeners are applied as interface in the code.

Listeners interfaces

ISuiteListener

- Works at suite level
- It listens and runs before the start and after the end of suite execution
- Abstract methods:
 1. onStart()
executes before the @BeforeSuite
 2. onFinish()
executes after the @AfterSuite

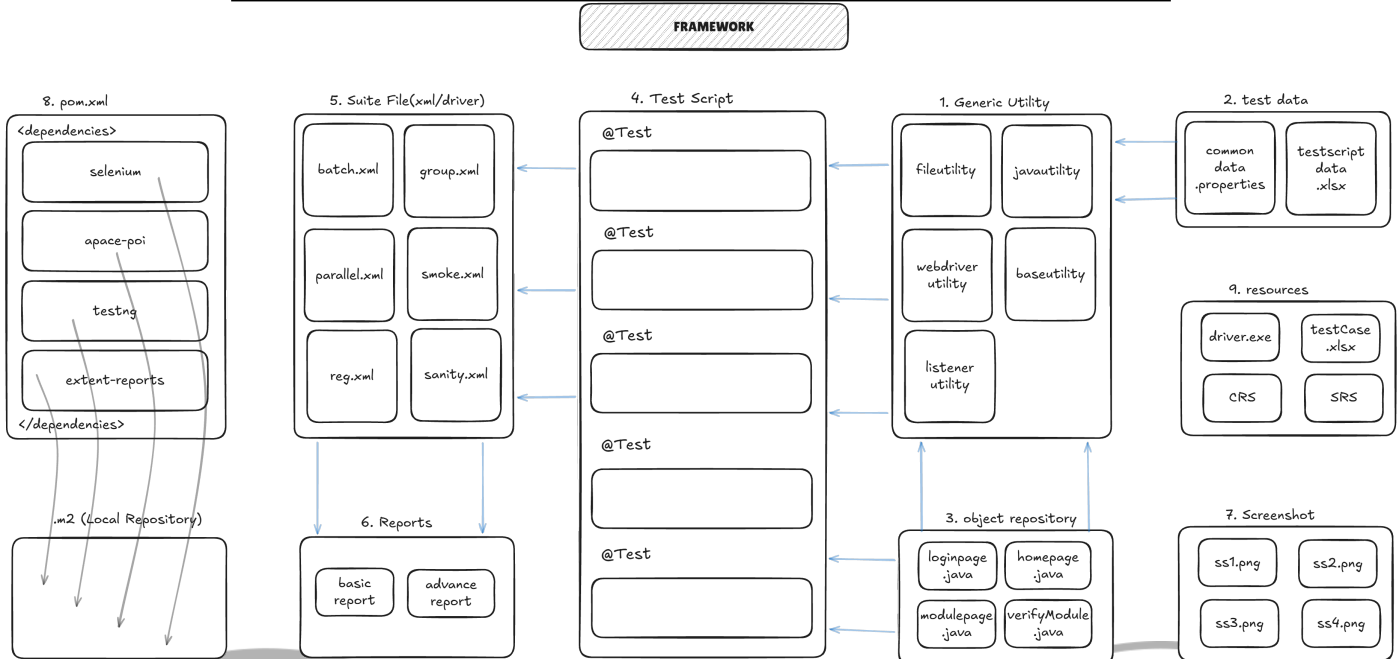
ITestListener

- Most frequently used testing listener
- Abstract methods:
 1. onTestStart()
 2. onTestSuccess()
 3. onTestSkipped()
 4. onTestFailure()

IRetryAnalyzer

- It helps user to retry the test script whenever test script is getting failed.
- In order to use this feature we have to implement IRetryAnalyzer interface and override retry().
- In retry(), we should specify the upper limit, so that test script will get executed till specified limit when the test is getting Failed again and again

FRAMEWORK EXPLANATION



1. Generic Utility.
2. Test data
3. Object Repository
4. Test Scripts.
5. Test Suite
6. HTML Report.
7. Screenshots.
8. pom.xml
9. Resources

Definition

- **Framework:** A framework is a well-organized structure of reusable components where one driver file will take care of the execution without any manual intervention.

“In my project, I have worked on a **hybrid Selenium automation framework using Java, Selenium, TestNG, Maven, and Page Object Model**.

The main objective of the framework was to make the automation **reusable, maintainable, scalable, and easy to integrate with CI/CD**.

Let me explain the framework structure and execution flow.

1. Generic Utility

First, we have the **Generic Utility** package.

This package contains all the common and reusable methods that can be used across multiple test cases.

For example, we have a **Base Class** where we manage browser initialization, launching the application, and closing the browser. We use TestNG annotations such as `@BeforeMethod` and `@AfterMethod` for login – logout and `@BeforeClass` and `@AfterClass` for setup and teardown.

We also have a **WebDriver Utility**, where we keep common Selenium operations such as handling waits, alerts, frames, windows, dropdowns, and other browser-related operations.

Apart from that, we have utilities for **Java operations, Properties files, and Excel files**.

The main purpose is reusability. If I have already written a common method once, I don't want to write the same logic again in every test script. I simply call that utility method wherever required.

2. Test Data

The second important component is **Test Data**.

The data which is common for all the test scripts is called as common data, and is stored in JSON file

The data which is common for particular test scripts is called as test script data, and is stored in excel file

3. Object Repository

Next is the **Object Repository**.

For this, we follow the **Page Object Model, or POM** design pattern.

Basically, every important page of the application has its own Java class.

For example, we can have a `LoginPage`, `HomePage`, `CustomerPage`, and `ProductPage`.

Inside these Page Object classes, we maintain the **locators and page-level actions** or business utility for example `login()`

The biggest advantage is maintainability.

4. Test Scripts

After that, we have the **Test Scripts**.

Our actual test scenarios are implemented using **TestNG classes**.

For example, we can have separate test classes for login, customer management, product management, or order management.

We use TestNG features such as **@Test, groups, priorities, dependencies, DataProvider, listeners, and parallel execution**, depending on the requirement.

The test classes mainly contain the **business flow**.

They call methods from the Page Object classes and Generic Utilities instead of hardcoding data and locator, it keeps our test scripts clean and readable.

5. Test Suite

Then we have the **Test Suite**.

We use **TestNG XML files** to control which tests need to be executed.

For example, we can create different suites for:

- Smoke testing
- Sanity testing
- Regression testing
- Cross-browser testing

Instead of manually selecting every test case, we can execute the required test suite through the TestNG XML file.

We can also configure things such as **test classes, groups, browsers, and parallel execution** through the suite configuration.

6. HTML Report

For reporting, we generate **HTML reports** after test execution.

We have 2 types of reports, basic and advance reports, basic reports will be auto generated once the testng is integrated with the framework, and advance report we need to configure using third party tool such as extent report or allure report, I have used extent report library for Advance reporting in my framework.

The report gives us a clear overview of the execution, such as which test cases **passed, failed, or were skipped**.

7. Screenshots

We have also implemented **screenshot capture for failed test cases**.

For this, we can use a **TestNG Listener**, particularly `ITestListener`.

When a test case fails, the listener triggers the screenshot-capturing logic.

The screenshot can then be attached to the report or can be stored in the screenshot folder component

8. pom.xml

Next is the **pom.xml** file.

Since our project is Maven-based, we use **pom.xml** for **dependency management and maven build lifecycle**.

For example, dependencies related to **Selenium, TestNG, Apache POI, reporting libraries**, and other required libraries are maintained in the **pom.xml**.

Maven also helps us execute the project from the command line and makes it easier to integrate our automation framework with **Jenkins for CI/CD**.

9. Resources

Finally, we have our **Resources**.

At the project level, we maintain a separate **Resources folder** for supporting project documentation and manual testing-related files.

For example, we keep:

- **Manual Test Cases in Excel format**
- **SRS — Software Requirements Specification**
- **CRS — depending on the project's documentation terminology**
- **Driver executable files**, such as `.exe` files when required
- Other project-related supporting documents

The reason for maintaining these separately is to keep the automation execution resources separate from the **project documentation and supporting files**.

Overall Framework Flow

The main advantages of our framework are **reusability, maintainability, readability, scalability, and reduced code duplication**.

overall, the framework follows **separation of concerns**, where test logic, page elements, reusable utilities, test data, execution configuration, reporting, and project documentation are maintained separately.

That makes the framework easier to maintain and scale as the application grows.”

Framework Advantages

1. **Saves cost & effort** – Less rework, more return on investment.
2. **Covers more tests** – Increases efficiency and test coverage.
3. **Run on any environment** – Works on Testing, UAT, SIT, etc.
4. **Fast test writing** – Reusable parts make scripts quicker to build.
5. **Easy to maintain** – Data & elements stored in one place (POM + external files).
6. **Test anywhere** – Cross-browser, distributed, and agile-friendly.
7. **Auto screenshots** – Captures failures for quick debugging.
8. **Clear reports** – Gives structured and accurate results.
9. **Handy utilities** – Excel, Base class, Listeners, etc. ready to use.
10. **Reusable scripts** – Same test can work across builds.

Types of Frameworks

1. Data driven Framework
2. Modular driven framework

3. Test driven development framework
4. Hybrid Driven framework
5. Keyword Driven framework
6. Behaviour driven development framework
7. (BDD)

1. Data-Driven Framework (DDF)

- **Includes 9 components:**
 1. Generic Utility
 2. Object Repository
 3. Test Data
 4. Test Script
 5. Resources
 6. Suite File
 7. Screenshot
 8. HTML Reports
 9. pom.xml

Key Features

- Simplifies test data maintenance.
- Uses `@DataProvider` annotation.
- One test script can run with multiple data sets.
- Each test script has its own Data-Provider or Excel sheet.

When to Use?

- When large amounts of data need to be tested.
- Examples: E-commerce, Banking, Booking applications.

Pros

- Reduces the number of test scripts.
- Allows testing multiple scenarios with minimal code.

Cons

- Requires skilled testers, proficient in programming to design and maintain the framework.

2. Modular-Driven Framework (MDF)

When to Use?

- For large applications with multiple modules (e.g., CRM systems).

Key Features

- Application is divided into small modules.
- Test scripts are created for each module independently.

Pros

- Easier to manage and scale.
- Independent test scripts for each module.

Cons

- Maintaining consolidated reports for scripts is time-consuming.
- Not preferred in modern agile terms.

3. Keyword-Driven Framework (KDF)

When to Use?

- When manual testers need to write automation scripts with minimal coding knowledge.
- Best for small applications (below 100 test cases).

Key Features

- Test cases are written in Excel.
- Keywords map to key generic libraries (e.g., `openBrowser()`, `ClickElement()`, `EnterText()`).
- Not Suitable for **large** CRM-type applications bcz it **slow down** execution for **thousands of test cases**.

TC_ID	StepDescription	Keyword	LocatorType	LocatorValue	TestData
TC_01	Open URL	OpenBrowser()	URL	https://example.com	
TC_01	Enter username	EnterText()	id	username	John
TC_01	Enter password	EnterText()	id	password	12345
TC_01	Click login button	ClickElement()	xpath	//button[@id='login']	
TC_01	Close browser	CloseBrowser()			

```
public class Keywords {
```

```
    WebDriver driver;
```

```
    public void openBrowser(String url) {
```

```
        driver = new ChromeDriver();
```

```
        driver.manage().window().maximize();
```



```
driver.get(url);
}

public void enterText(String locatorType, String locatorValue, String testData) {
    getElement(locatorType, locatorValue).sendKeys(testData);
}

public void clickElement(String locatorType, String locatorValue) {
    getElement(locatorType, locatorValue).click();
}

public void closeBrowser() {
    driver.quit();
}

private WebElement getElement(String locatorType, String locatorValue) {
    switch(locatorType.toLowerCase()) {
        case "id": return driver.findElement(By.id(locatorValue));
        case "name": return driver.findElement(By.name(locatorValue));
        case "xpath": return driver.findElement(By.xpath(locatorValue));
        case "css": return driver.findElement(By.cssSelector(locatorValue));
        default: throw new RuntimeException("Invalid locator type: " + locatorType);
    }
}
}
```

```
public class TestScript {
```

```
    public static void main(String[] args) throws Exception {
        FileInputStream fis = new FileInputStream("TestSteps.xlsx");
        Workbook workbook = WorkbookFactory.create(fis);
    }
}
```

```

Sheet sheet = workbook.getSheet("sheetName");

Keywords keywords = new Keywords();

for (int i = 1; i <= sheet.getLastRowNum(); i++) {

    Row row = sheet.getRow(i);

    String keyword = row.getCell(2).getStringCellValue();

    String locatorType = row.getCell(3).getStringCellValue();

    String locatorValue = row.getCell(4).getStringCellValue();

    String testData = row.getCell(5).getStringCellValue();

    switch(keyword) {

        case "OpenBrowser": keywords.openBrowser(locatorValue); break;

        case "EnterText": keywords.enterText(locatorType, locatorValue, testData); break;

        case "ClickElement": keywords.clickElement(locatorType, locatorValue); break;

        case "CloseBrowser": keywords.closeBrowser(); break;

        default: System.out.println("Invalid keyword: " + keyword);

    }

}

workbook.close();

}

}

```

Pros

- Easy maintenance and reusability.
- Non-coders can contribute via Excel.
- Makes automation highly modular and flexible.
- Clear separation between **test logic**, **test data**, and **test script**.

Cons

- **Slower execution** due to reading from external files.
- **Harder debugging** if keywords or mapping logic fails.
- Requires good **framework design discipline**, else it becomes a mess of keywords.

- Poor performance with **large datasets**.

Framework Development Stages

Automation frameworks are not built all at once—they go through **3 main stages**. Think of it like building a house: first you plan, then you build, and finally you live in it and use it!

1. Framework Design & Implementation (Planning and Setup Phase)

When? – Usually done in **Sprint 1** (the first few days/weeks of the project)

Who? – Experienced Automation Test Engineers (7 – 9 years)

What happens here?

- Decide **which tools** to use (like Selenium, TestNG, Maven, Jenkins).
- Create basic **utility classes** (e.g., for browser handling, data reading).
- Set up the **project structure** and folders.
- Create a small working model (called **POC – Proof of Concept**) for important features.
- Show this POC to the client or manager to get their approval.

Goal: Build the base or skeleton of the framework.

2. Framework Test Development (Building Phase)

When? – Usually in **Sprint 2** and continues through development

Who? – Automation Test Engineers (3 – 5 years)

What happens here?

- Start writing **real automation scripts** using the framework.
- Convert **manual test cases** into automation code.
- Store and organize all scripts in version control tools like **GitHub**.
- Review the code with teammates or seniors to ensure quality.

Goal: Fill your framework with working, reviewed, and reusable test scripts.

3. Framework Execution (Using and Monitoring Phase)

When? – Happens regularly after scripts are ready (during every release or sprint)

Who? – Mostly Senior Automation Test Engineers or DevOps team

What happens here?

- Test scripts are run using **Jenkins** or other CI tools (can be scheduled).
- After execution, an **HTML report** is generated showing pass/fail/skipped.
- If any script fails, screenshots are automatically captured.
- Engineers do **debugging**, **fix issues**, and **retest**.

Goal: Run the tests **smoothly**, find bugs, and provide reports to the team or client.

Summary:

Stage	Name	Main Tasks	Who Does It
1	Design & Implementation	Set up tools, create POC, write base classes	Senior Automation Engineers
2	Test Development	Write and review automation scripts	Automation Engineers
3	Execution	Run tests, report results, fix issues	Senior Testers or DevOps



Maven

1. What is Maven?

- **Apache Maven** is a **build automation and framework management tool**.
- Another framework management tools are:
 - **Maven**
 - Gradle
 - Ant
- It is mainly used for **Java projects** and helps manage:
 - **Dependencies (JARs/Libraries)**
 - **Project structure(4 Folder structure)**
 - **Build lifecycle**

2. Why Maven in Selenium Automation?

When you're working on **Selenium projects**, you'll need several libraries like:

- selenium-java
- testng or junit
- apache poi (for Excel), etc.

⚡ Without Maven: You download and add these libraries manually.

⚙ With Maven: You **just declare dependencies in pom.xml**, and Maven automatically downloads everything.

✓ 3. Key Maven Concepts

a) POM (Project Object Model) File – pom.xml

- XML file that defines:
 - Project info (name, version)
 - Dependencies (libraries like Selenium)
 - **Build lifecycle**

b) Dependencies

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId> ➔ organization name
  <artifactId>selenium-java</artifactId> ➔ project name
  <version>4.27.0</version>
</dependency>
```

c) Repositories

- Central: Maven Central Repository (default source for downloading jars)
<https://www.mvnrepository.com>
- Local: Your computer's **.m2** folder (stores downloaded jars)

✓ 4. Maven Directory Structure

```
project-name/
├── src/
│   ├── main/java
│   ├── main/resource
│   ├── test/java
│   └── test/resource
└── pom.xml (Maven configuration file)
```

✓ 5. Maven Commands

Command	Purpose
mvn clean	Deletes target/ folder (cleans project)
mvn validate	Checks project setup, POM, directories, dependencies.
mvn compile	Compiles the Java code
mvn test	Runs the test cases
mvn package	Creates the .jar file
mvn install	Installs build into the local repo

✓ 6. Real-World Use in Java Selenium

- Create a **TestNG framework** with:
 - Selenium dependency
 - TestNG dependency
- Run tests with **Maven commands or Jenkins**

✓ 7. Benefits for Automation Engineers

Feature	Benefit
Dependency Mgmt	No manual jar handling
Standard structure	Clean and manageable projects
Easy integration	Works well with Jenkins, Docker, CI/CD
Scalability	Helpful in big projects with multiple modules

What are the Maven Software required to handle maven Project in real time?

⇒ Two software available in Maven

1. Maven Eclipse Plugin

➔ Maven Eclipse Plugin: it's a default plugin available in Eclipse IDE, used to handle maven project in Eclipse

2. Maven Command Line Plugin

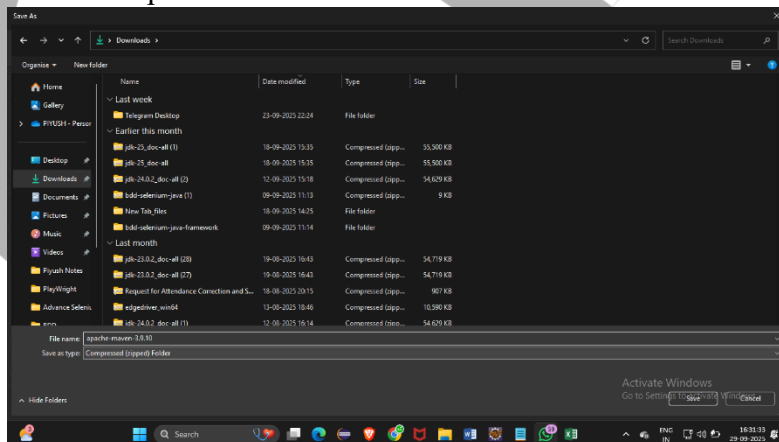
➔ Maven Command Line Plugin: it's should be installed explicitly in all local computers used to handle the Maven project in CMD line

How to extract zip folder?

1. Download the zip folder by this link

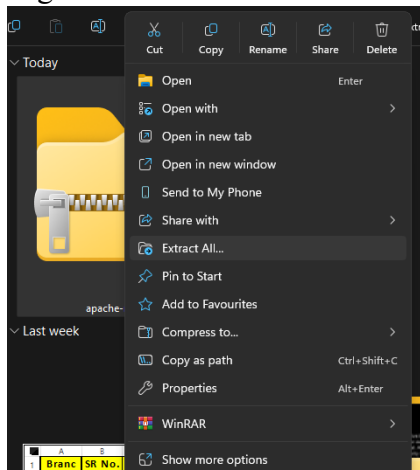
<https://d1cdn.apache.org/maven/maven-3/3.9.16/binaries/apache-maven-3.9.16-bin.zip>

2. Save the zip folder in downloads folder



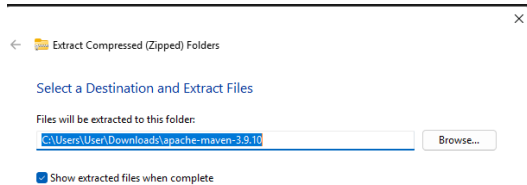
3.

4. Right click on where it is downloaded and select extract all

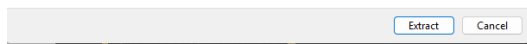


5.

6. Click on extract button

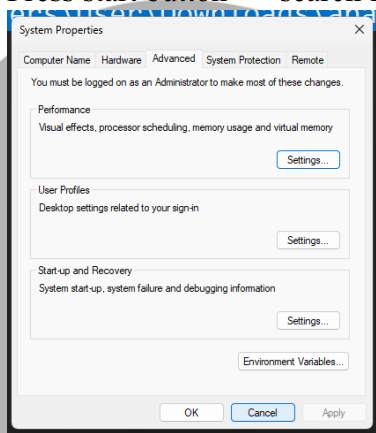


7.



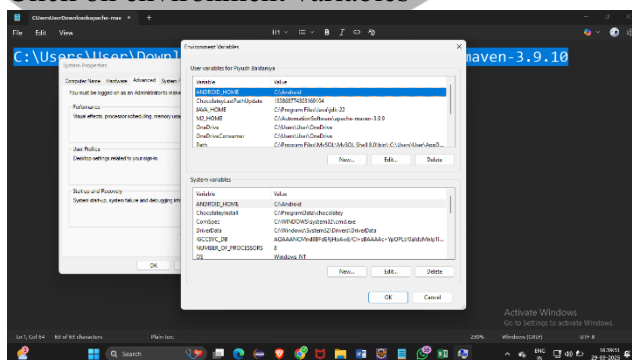
How to set it as system and environment variable?

1. Copy 2 paths (There should not be .zip in between)
C:\Users\User\Downloads\apache-maven-3.9.10\apache-maven-3.9.10
C:\Users\User\Downloads\apache-maven-3.9.10\apache-maven-3.9.10\bin
2. Press start button => search for "edit the system environment variable"



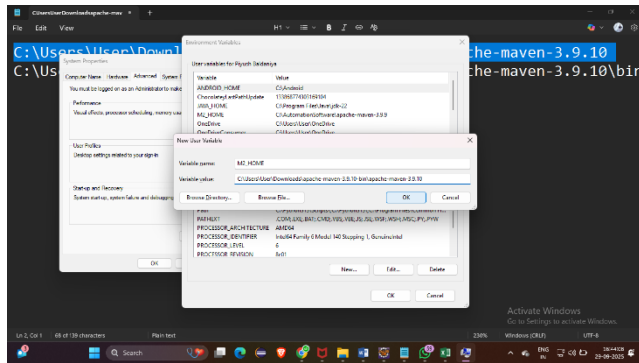
3.

4. Click on environment variables

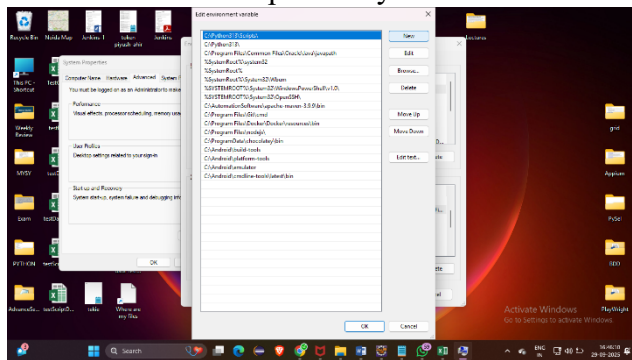


5.

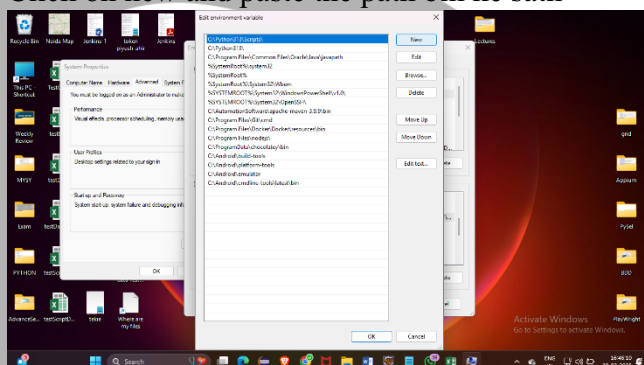
6. Click on new (upar vala) and paste the path (BIN ke bina) and press ok



- 7.
8. And then search for path in system variables and double click on that



- 9.
10. Click on new and paste the path bin ke sath



11. And then ok => ok => ok

How to check whether it got installed or not?

1. Press windows key + R => type cmd => ok
2. Type mvn -v and press enter

```
C:\Users\User>mvn -v
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfc9d97d260186937)
Maven home: C:\AutomationSoftware\apache-maven-3.9.9
Java version: 22.0.2, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-22
Default locale: en_IN, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

- 3.

How to open the project in cmd?

1. Right click on project => properties => copy the location

2. Open command prompt => write

```
C:\Users\User>cd C:\Users\User\Basic_Selenium\maven-project-a11
```

3. So you will be having

```
C:\Users\User\Basic_Selenium\maven-project-a11>
```

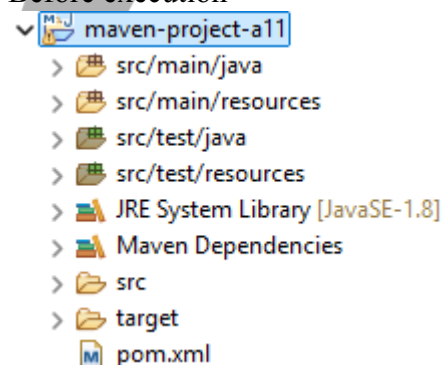
4. Run it with mvn test

```
C:\Users\User\Basic_Selenium\maven-project-a11>mvn test
```

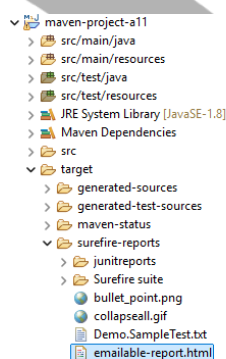
5. You will get result

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.436 s  
tSuite  
[INFO] Results:  
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0  
[INFO] -----  
[INFO] BUILD SUCCESS  
[INFO] -----  
[INFO] Total time: 2.099 s  
[INFO] Finished at: 2025-09-30T16:32:41+05:30  
[INFO] -----
```

Before execution



After execution



How to run particular one testng class?

```
C:\Users\User\Basic_Selenium\maven-project-a11>mvn -Dtest=DemoTest test
```

How to run particular one test method?

```
C:\Users\User\git\MavenJenkinsM1\MavenProjectM1>mvn -Dtest=DemoTest#case1 test
```

What is profiling? when is it required?

➔ To execute specific suite file from multiple suite files.

In pom.xml we create profiles like this,

```
<profiles>
  <profile>
    <id>smoke</id>
    <build>
      <plugins>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.5.4</version>
          <configuration>
            <suiteXmlFiles>
              <suiteXmlFile>smoke.xml</suiteXmlFile>
            </suiteXmlFiles>
          </configuration>
        </plugin>
      </plugins>
    </build>
  </profile>
  <profile>
    <id>regression</id>
    <build>
      <plugins>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.2.5</version>
          <configuration>
            <suiteXmlFiles>
              <suiteXmlFile>regression.xml</suiteXmlFile>
            </suiteXmlFiles>
          </configuration>
        </plugin>
      </plugins>
    </build>
  </profile>
</profiles>
```

➔ To trigger particular suite file

```
C:\Users\User\Basic_Selenium\maven-project-a11>mvn test -P smoke
```

GITHUB

On-Premises & Centralized Repositories

These tools are installed and maintained **within an organization's infrastructure**. They require dedicated hardware and a maintenance team.

- **SVN (Subversion)**
 - **CVS (Concurrent Version System)**
 - **VSS (Visual SourceSafe)**
 - **Perforce**
-

Cloud-Based Repositories

These are **cloud-hosted platforms** where the source code can be stored and managed without on-premise maintenance.

- **GitHub**
 - **AWS CodeCommit**
 - **GCP Source Repositories**
-

What is GitHub? *(Source Code Management (SCM) / Code Management tool / Configuration Management tool / Version Control Tool)*

GitHub is a **cloud-based distributed (decentralized) repository** platform that allows you to manage your **source code, automation framework, project documentation (CRS), and build artifacts** in one centralized cloud location.

GitHub is based on **Git**, an open-source version control system.

There are **two key components** in the Git ecosystem:

1. **GitHub (Server-side software):**
A **cloud-based repository hosting service**, used to store and manage code.
To use it, simply create an account at <https://github.com>.
 2. **Git (Client-side software):**
A **local client application** that must be installed on your system to interact with GitHub.
Examples: **Git Bash, Egit, Git Desktop**.
-

Advantages of GitHub (Cloud-Based Repository)

1. **No Maintenance Required:**
No need for an internal maintenance team to manage software or hardware.
2. **Pay-what-You-Use:**
You only pay for the storage or services you consume.
3. **Accessible via Internet:**
GitHub can be accessed anytime, anywhere with an internet connection.
4. **No Physical Setup:**
The cloud server is hosted virtually, not inside your organization.
5. **No Initial Investment:**
No upfront cost for infrastructure or software licenses.
6. **Scalability:**
Easily scale up or down based on usage.
7. **Simplified Collaboration:**
Team members can easily share files and work collaboratively.
8. **Remote Access:**
Contributors can access and update repositories from anywhere.
9. **Version History & Backup:**
Keeps track of all code changes and provides rollback options.
10. **Code Review Support:**
Pull requests enable peer review and code approval before merging.
11. **Conflict Management:**
GitHub helps handle code merge conflicts efficiently.
12. **Integration with Jenkins:**
Jenkins can automatically fetch the latest code from GitHub for continuous integration and batch execution.

Usage of Git Across Teams

- **Developers:** Maintain the application's source code.
- **Automation Testers:** Maintain the complete automation framework.
- **DevOps Engineers:** Store and version multiple build artifacts (like .exe, .war, .jar, .tar, etc.).
- **Manual Testers:** Maintain documents like CRS (Customer Requirement Specification), test cases, and use cases.

Git Architecture

Git follows a **decentralized architecture**, meaning every developer has a full copy of the repository (including its history) on their local system.

Before pushing code to the **remote (GitHub) repository**, the developer first commits it to the **local repository**. This ensures that the code is verified and tested locally before being shared globally.

Three Stages in Git Workflow

1. **Working Directory:** Area where you modify files.
2. **Local Repository:** Where changes are committed after local verification.
3. **Remote Repository (Global Repo):** The GitHub repository where code is pushed for collaboration

Can u explain Git Commands?

1. Commit
2. Pull
3. Push
4. Merg
5. Clone
6. Fork

To push the framework to the github we required 3 things

1. URL
2. Username
3. Token / Password

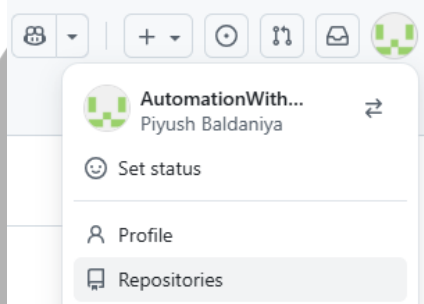
Pushing

i. Create account with Git-Hub

1. Go to www.github.com
2. Click on sign Up
3. Create an account [capture username/password]

ii. Create Repository in Git-Hub

1. Login to GitHub and go to profile icon



2. click on “repositories” and click on “+” or
3. enter ONLY name



1 General

Owner *

AutomationWithPiyushhh

Repository name *

MavenJenkinsM1

✓ MavenJenkinsM1 is available.

4. Click on “create repository” BUTTON
5. Copy the URL and paste it in **notepad**

Quick setup — if you've done this kind of thing before

Set up in Desktop or HTTPS SSH <https://github.com/AutomationWithPiyushhh/MavenJenkinsM1.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

URL: <https://github.com/AutomationWithPiyushhh/MavenJenkinsM1.git>

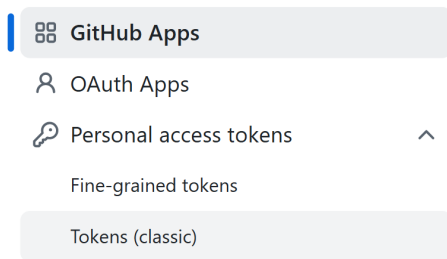
Username: AutomationWithPiyushhh

Token: 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

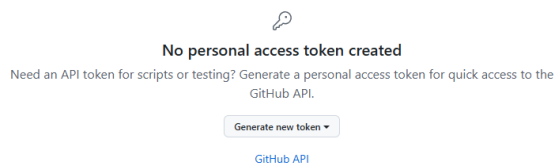
iii. Create Token ID

1. login to GitHub

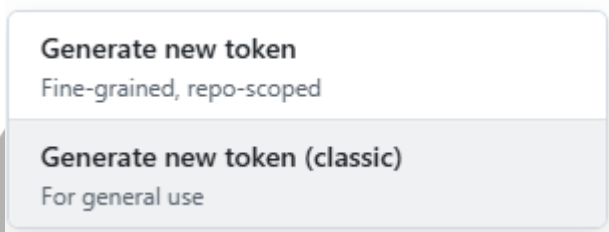
2. Go to “Profile Image” Icon
3. Click on Settings
4. Click on “Developers settings”
5. Click on “Personal Access Token”



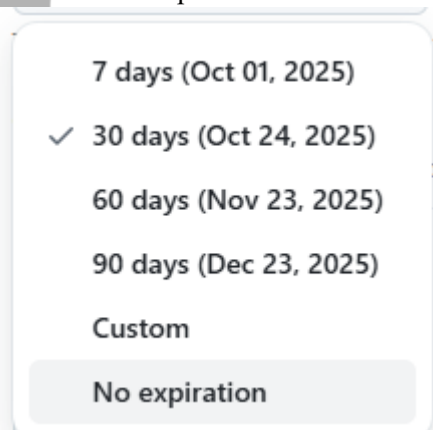
- 6.
7. Click on “Generate new Token” button



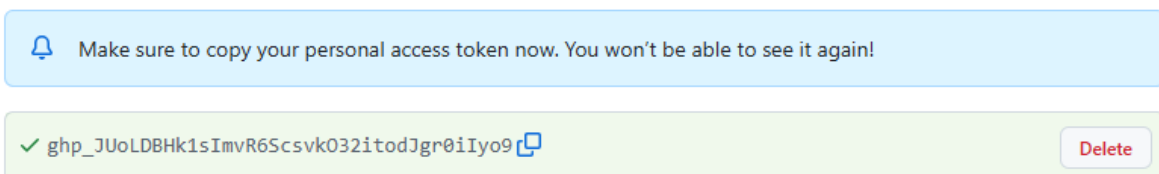
- 8.



- 9.
10. Click on No expiration

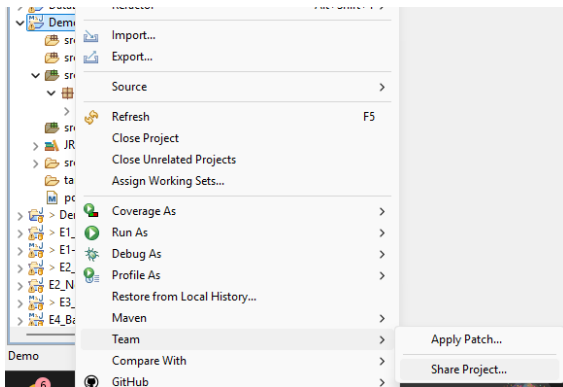


- 11.
12. Enter the note & select all the “Checkboxes”
13. Click on “generate token” button
14. Copy the Token ID



iv. Create local repository

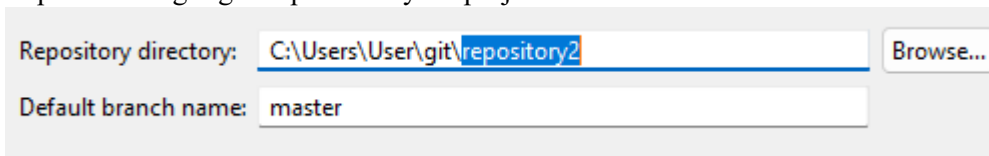
1. Select the Project → Right click → Team → Share Project



- 2.
3. Click on create button



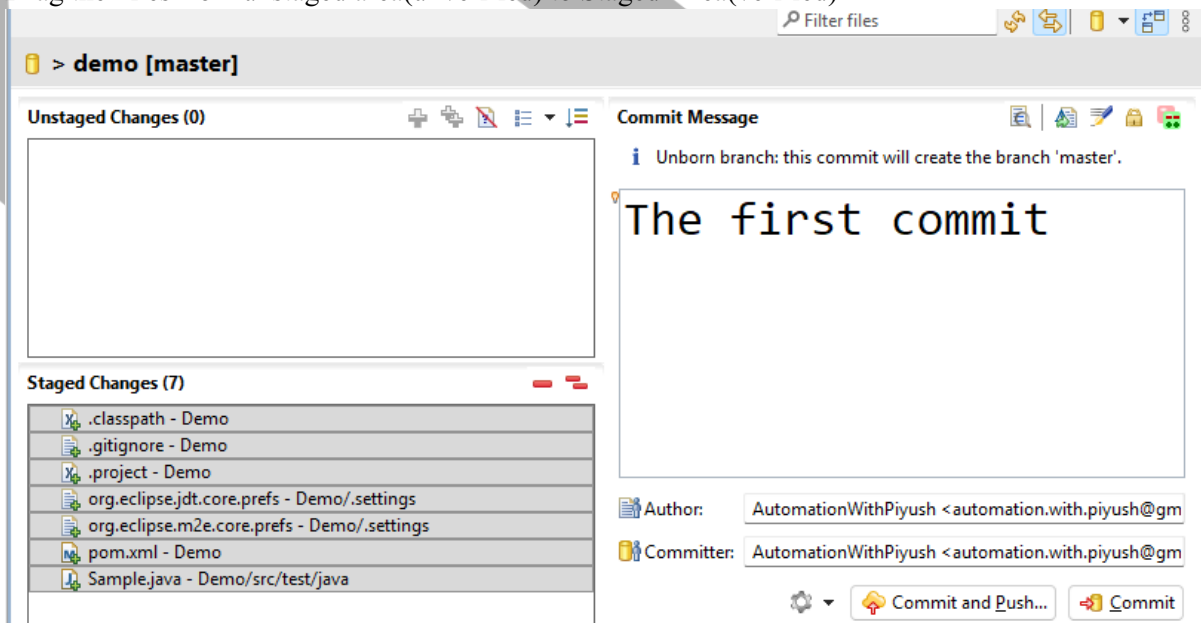
- 4.
5. Replace the highlighted part with your project name



6. Click on Finish and again finish

v. Commit from Working directory to Local Repository

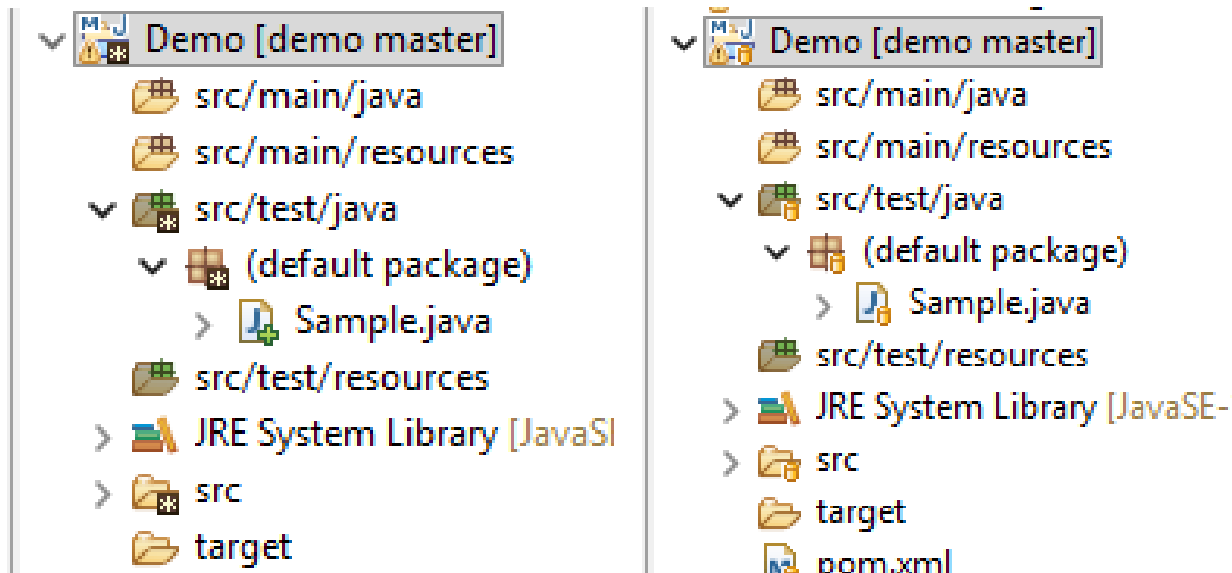
1. Select the Project → Right click → Team → Commit
2. Drag the Files from unstaged area(unverified) to Staged-Area(verified)



- 3.
4. Enter some commit message
5. Click on "Commit"

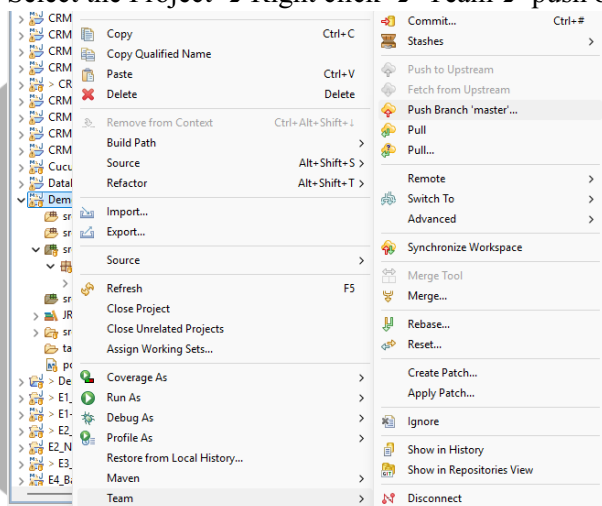
Before look

After look

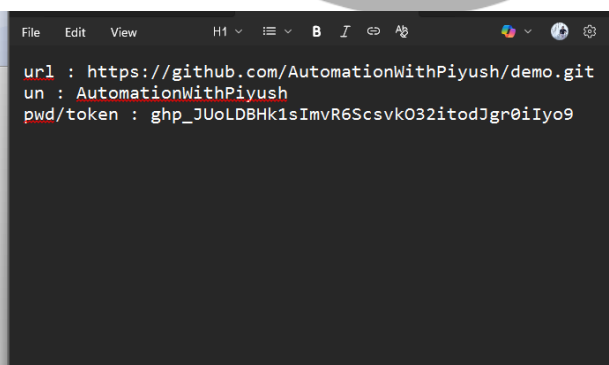
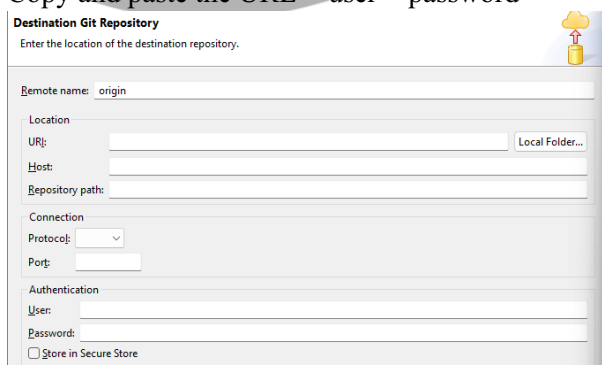


vi. Push from Local Repo to Global Repository

1. Select the Project → Right click → Team → push branch master



- 2.
3. Copy and paste the URL + user + password



5. Click on store checkbox

- 6.
7. Click on preview again preview and then push

Pulling

How to get Framework from global repository to local System very first Time?

1. Go to Eclipse
2. Click on “File”
3. Click on **import**
4. Expand “Git” folder
5. Click on “Project from git”
6. Click on “Clone URL”
7. Enter “Git URL” username & Token ID
8. Click on finish

How to get newly created file from Global to local?

1. Select the project
2. Right Click → Team
3. Click on “pull” => merge
4. Click on finish

Git Conflicts

What is Git Conflicts?

When two or more engineers modified the same file, when Eng-1 one push the file to GitHub will not get any conflicts, but Eng-2 try to push the same file to Git-Hub, will get git Conflicts(because that file already modified by Eng-1)

Solution:

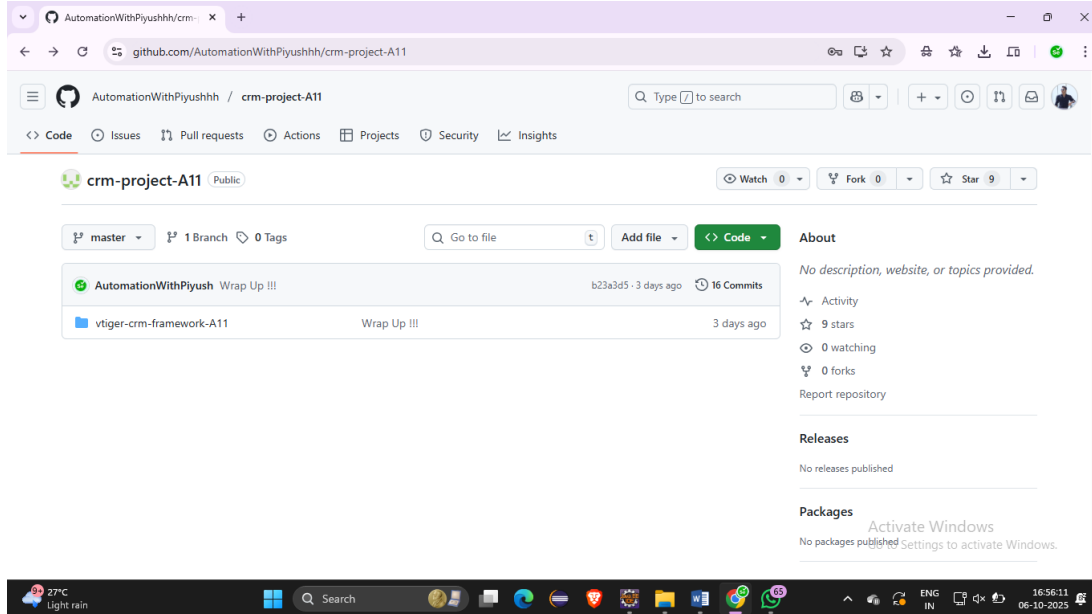
Before push any file to GitHub, we should mandatorily Pull the Project & select merge option , then[eclipse automatically merge the Code with comments]& remove comments manually & analyse the code then push it

What is Pull Request & git branching?

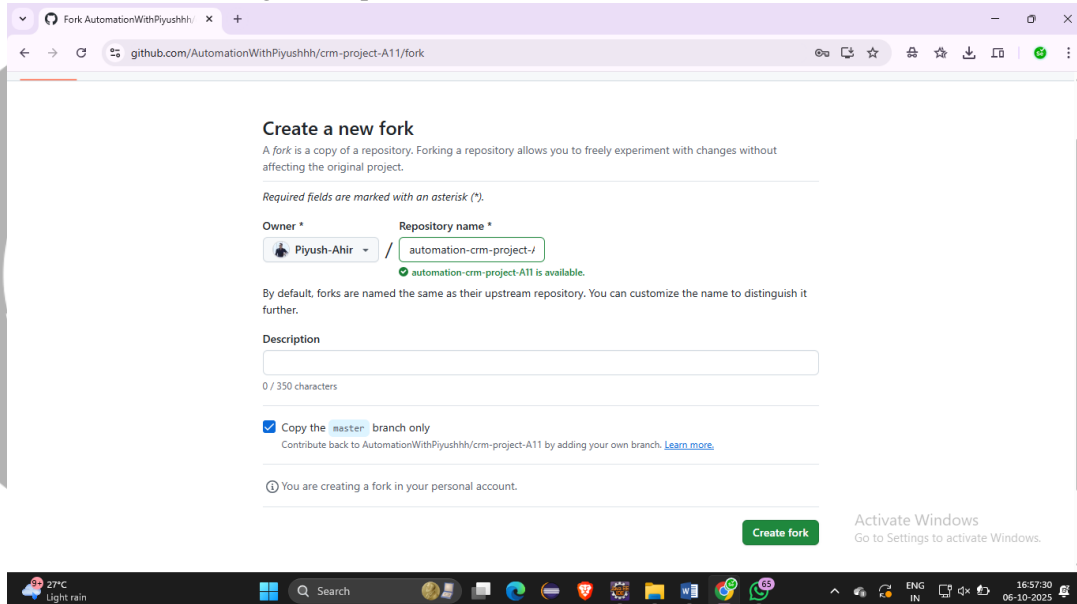
Whenever user made changes in the Programs , some Organization will not have access to push your changes to Master-Branch , in such case we supposed to create a local branch using “fork” option , than push the changes to local branch & than create “Pull Request” to Team lead asking for “REVIEW & MERGE” my program to master branch

Steps for Forking and Pull Request:

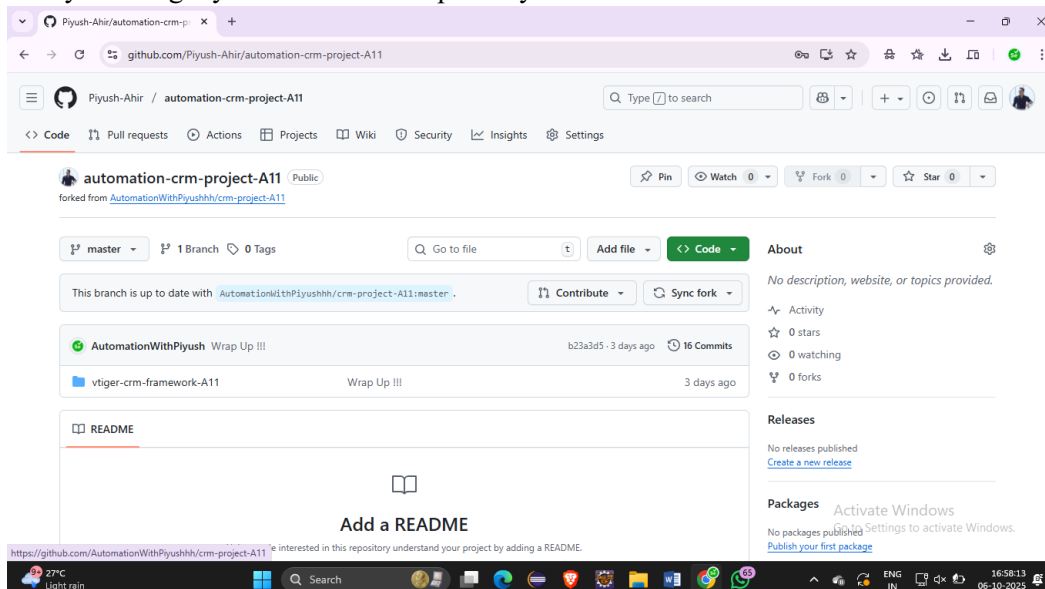
1. Go to the git URL



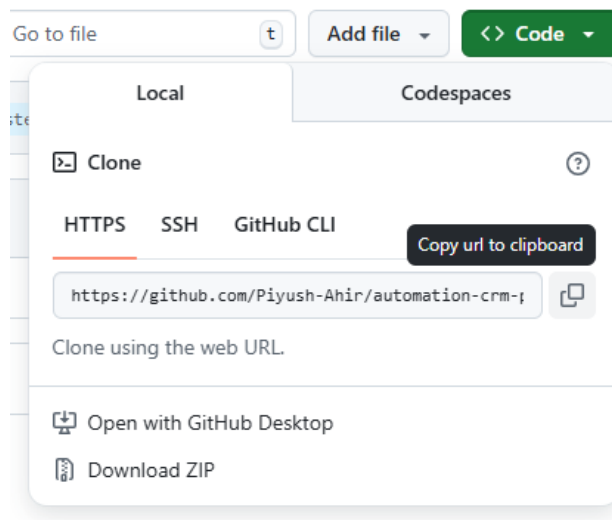
2. Click on fork -> change the repo name -> click on create fork button



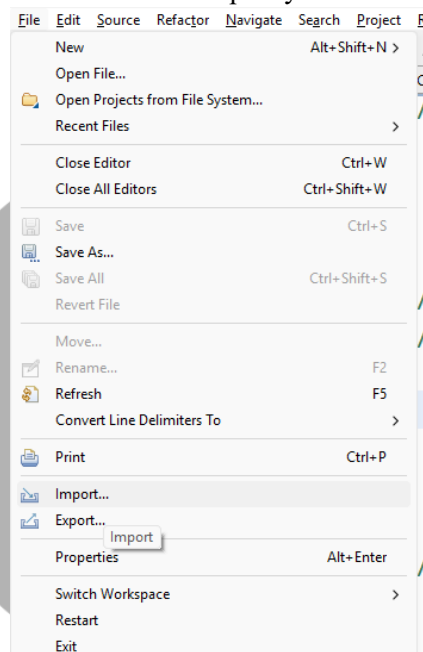
3. And you will get your own forked repository



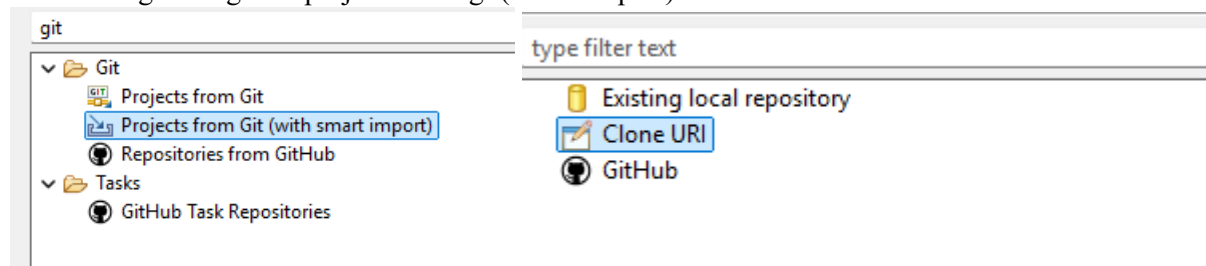
4. Copy the url of this repository



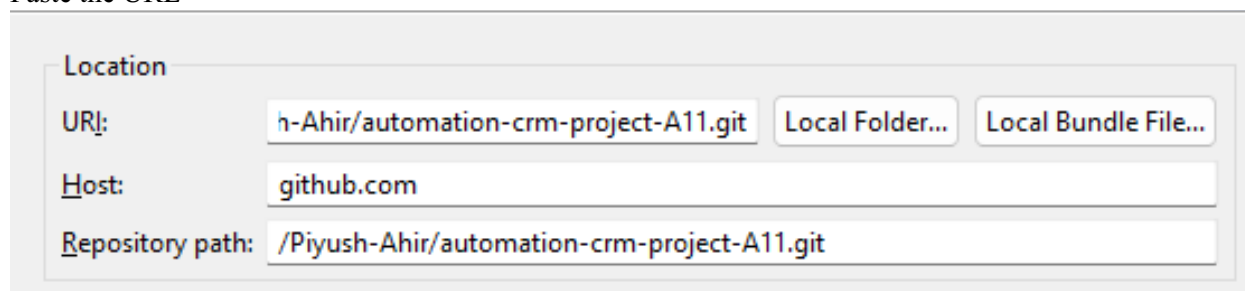
5. Now its time to import your forked repo



6. Search for git and go for projects from git(Smart import) and then clone URI

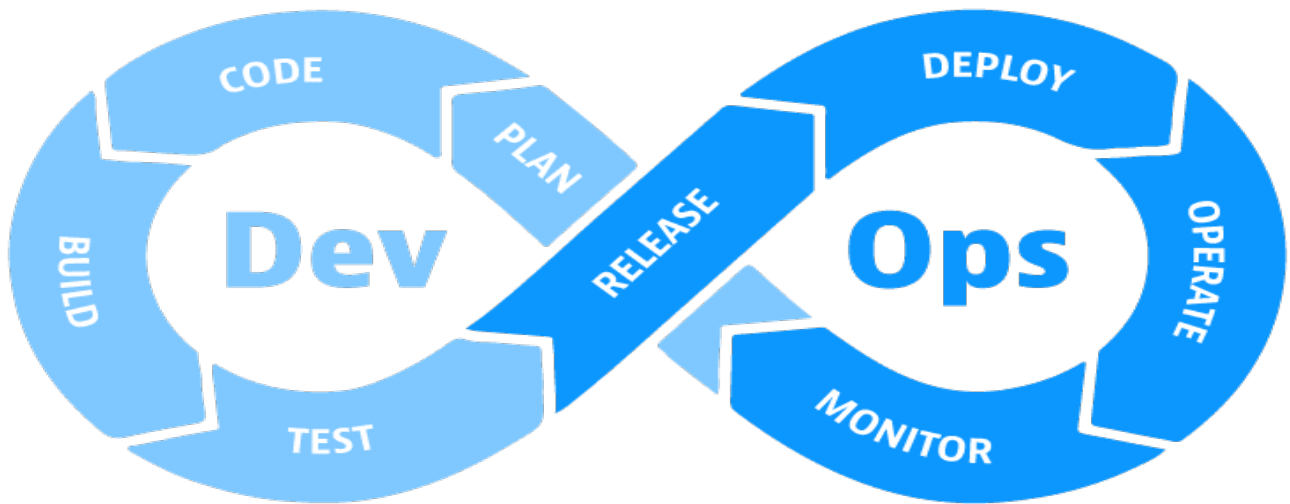


7. Paste the URL



8. Next -> next -> next -> finish

Jenkins



Why Jenkins is Required in Development?

Continuous Development: Continuously monitor the Git source code repository and create a new build if changes are detected.

Continuous Deployment: Retrieve the latest build from the Git repository and deploy it to the **testing environment**.

Continuous Delivery: Retrieve the latest build from the Git repository and deploy it to the **UAT environment**.

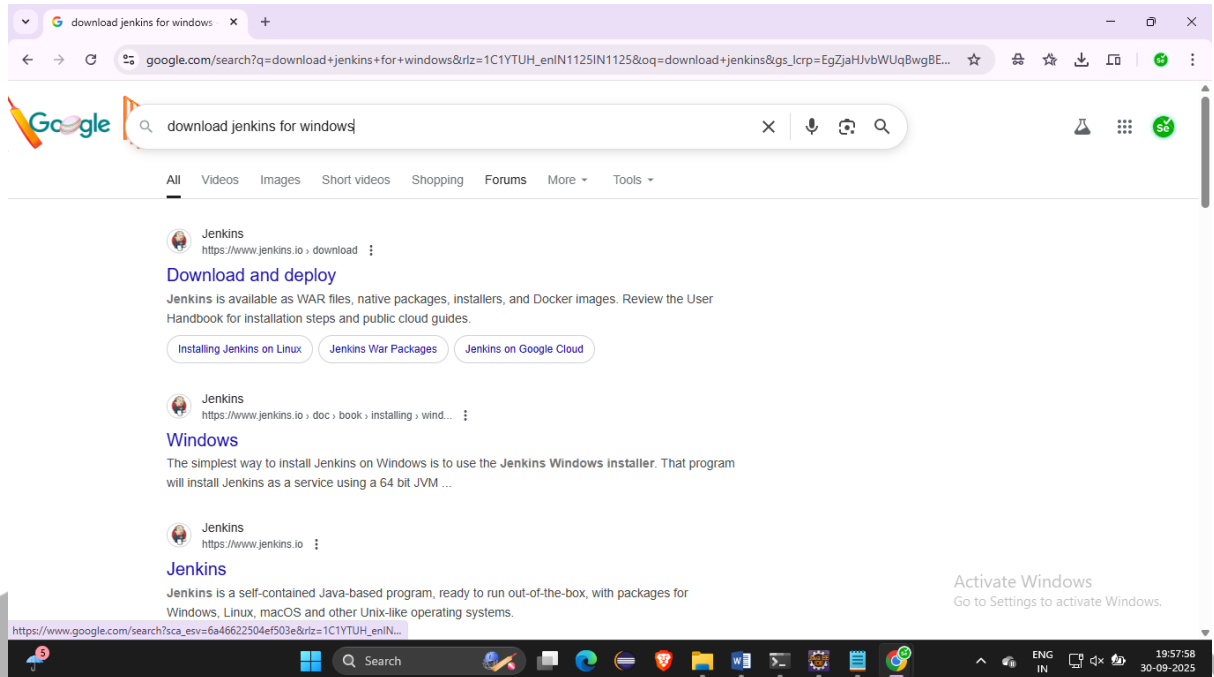
Why Jenkins is Required in Automation?

Continuous Integration: **Execute** Selenium test scripts in the **testing environment**.

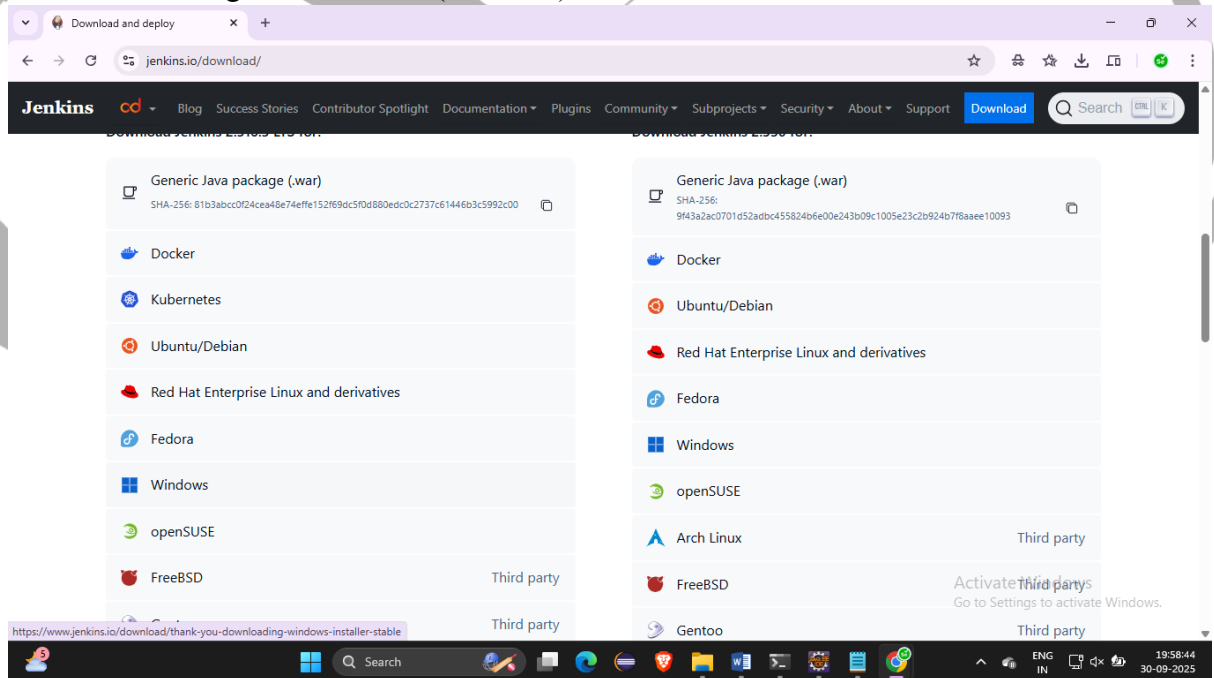
Installation Steps

a. Download and Install Jenkins

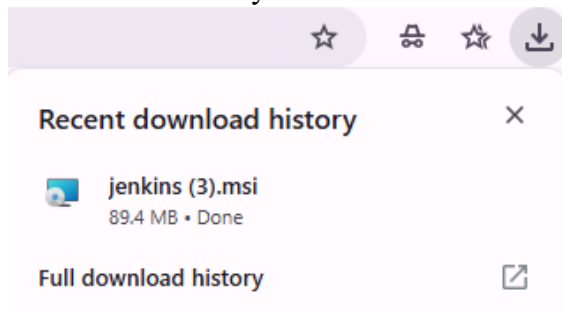
1. Search “download Jenkins for windows” and click on the first link



2. Scroll down and go for windows (Left side)



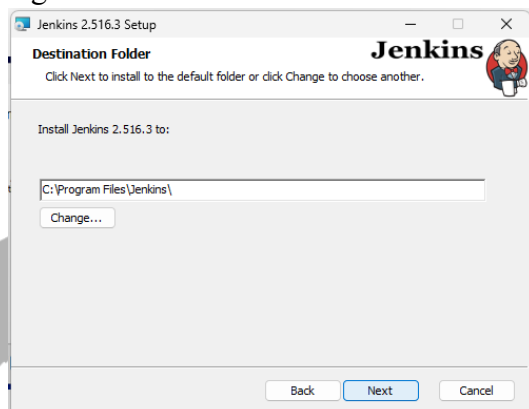
3. It will automatically download the Jenkins.msi



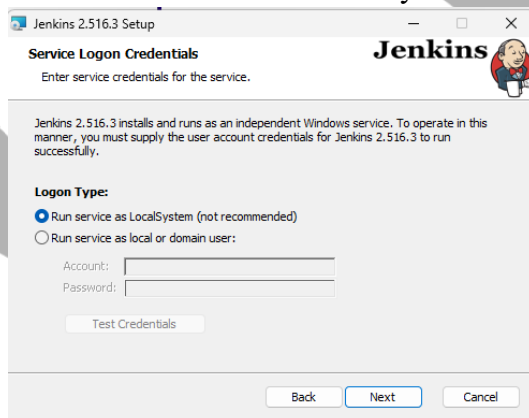
4. Open that msi file
5. Click on next



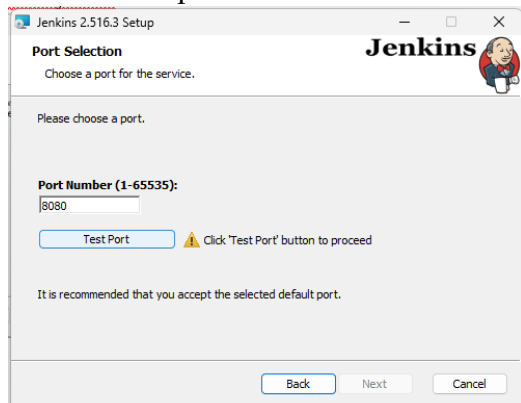
6. Again next



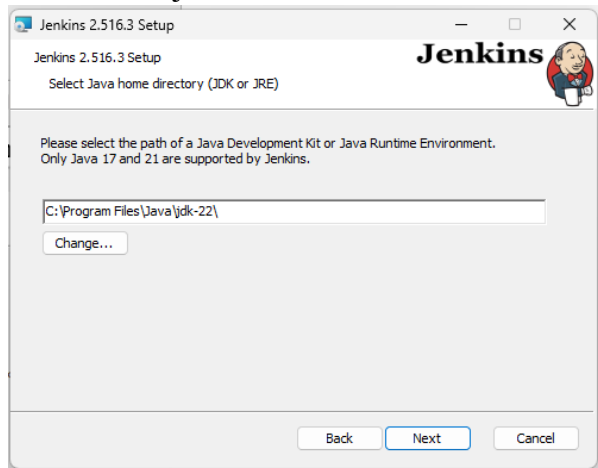
7. Click on run service as LocalSystem



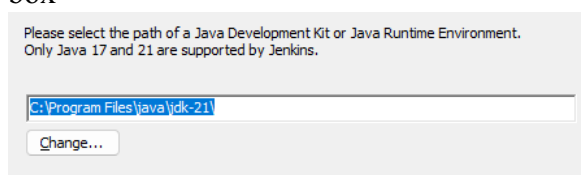
8. Click on test port and then next



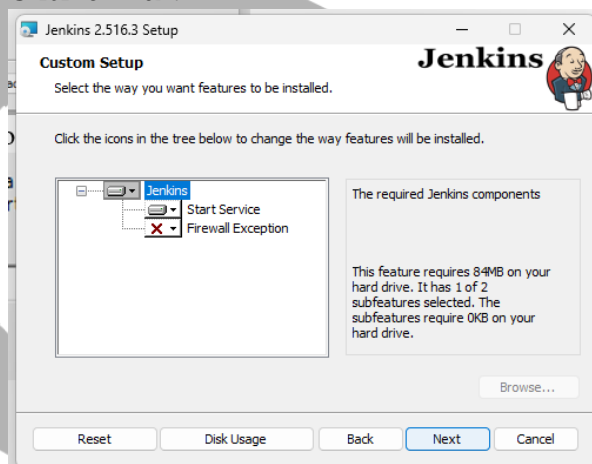
9. It should be jdk-21



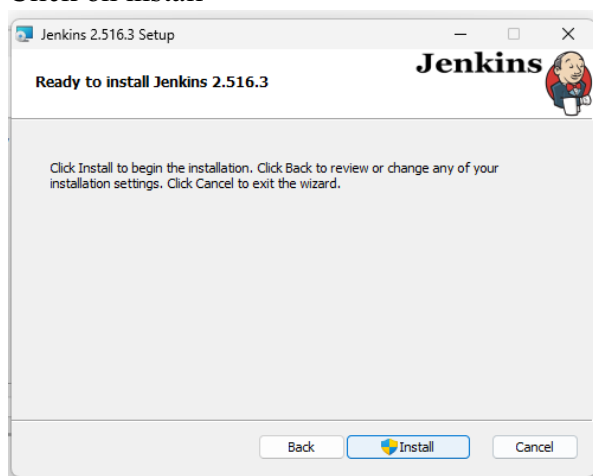
10. But I am having jdk-22 so need to change it, so download jdk-21 and give path of it to the text box



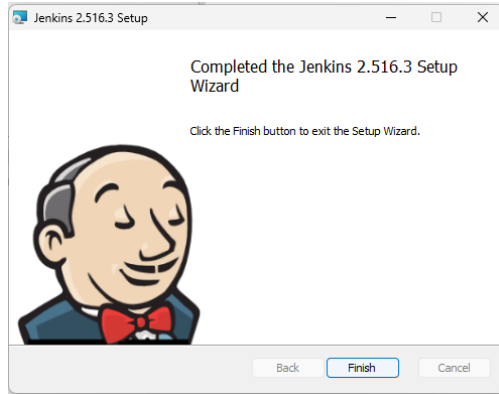
11. Click on next



12. Click on install



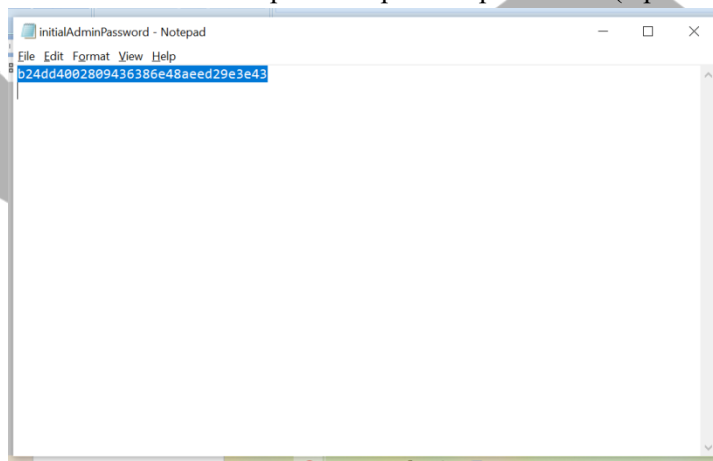
13. It will ask you for some permission, Give yes and you will get completion wizard, click on finish



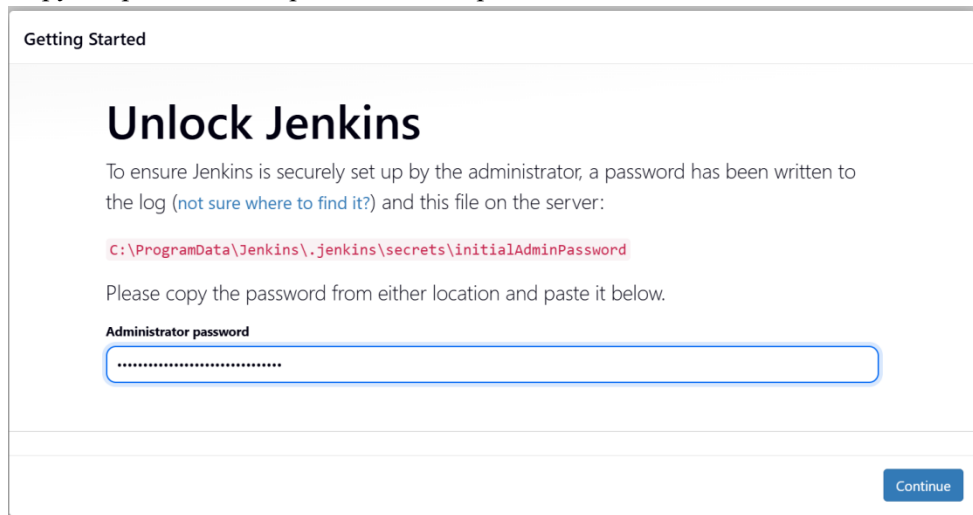
14. Go to google and write localhost:8080



15. Press window + R => paste the path => press enter (Open in notepad)



16. Copy the password and paste it to the input field and click on continue



The screenshot shows the 'Getting Started' page for Jenkins with the title 'Unlock Jenkins'. It explains that a password has been written to the log and the server file. The file path `C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword` is highlighted in red. Below this, it asks the user to copy the password and paste it into a text input field labeled 'Administrator password'. The input field contains a series of dots. A 'Continue' button is located at the bottom right.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

`C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword`

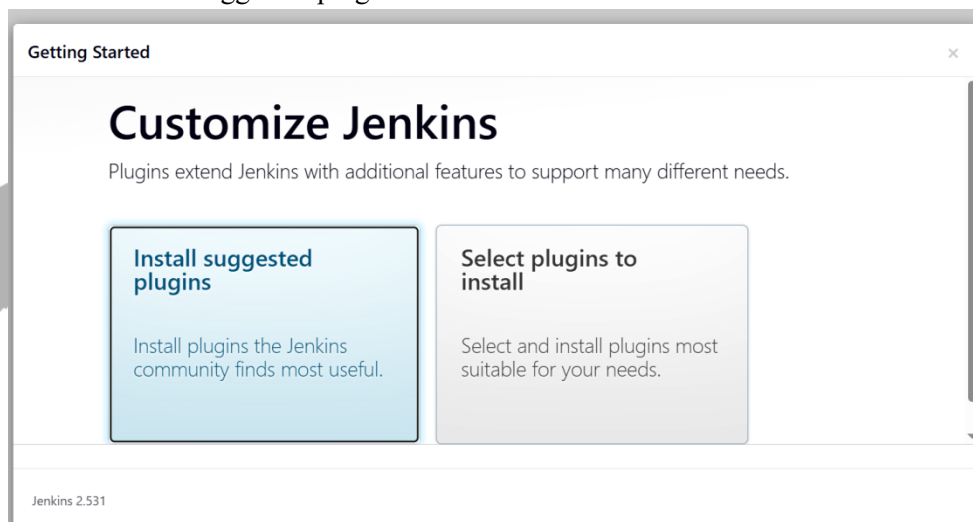
Please copy the password from either location and paste it below.

Administrator password

.....

Continue

17. Click on install suggested plugins



The screenshot shows the 'Getting Started' page for Jenkins with the title 'Customize Jenkins'. It explains that plugins extend Jenkins with additional features. There are two main options: 'Install suggested plugins' (highlighted with a blue border) and 'Select plugins to install'. The 'Install suggested plugins' option includes the text 'Install plugins the Jenkins community finds most useful.' The 'Select plugins to install' option includes the text 'Select and install plugins most suitable for your needs.' The Jenkins version 'Jenkins 2.531' is displayed at the bottom left.

Getting Started

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

Install suggested plugins

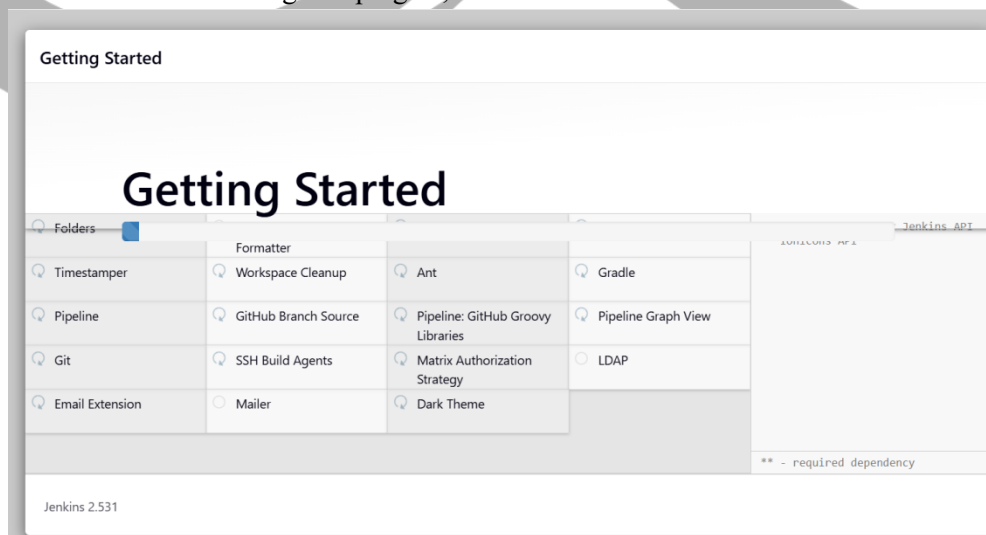
Install plugins the Jenkins community finds most useful.

Select plugins to install

Select and install plugins most suitable for your needs.

Jenkins 2.531

18. It will start downloading few plugins, let them downloaded



The screenshot shows the 'Getting Started' page for Jenkins with the title 'Getting Started'. It displays a list of plugins that are being installed or suggested. The plugins are organized into a grid. The 'Folders' tab is selected. The plugins listed are: Timestamp, Pipeline, Git, Email Extension, Formatter, Workspace Cleanup, GitHub Branch Source, SSH Build Agents, Mailer, Ant, Pipeline: GitHub Groovy Libraries, Matrix Authorization Strategy, Dark Theme, Gradle, Pipeline Graph View, and LDAP. A note at the bottom right states '** - required dependency'. The Jenkins version 'Jenkins 2.531' is displayed at the bottom left.

Getting Started

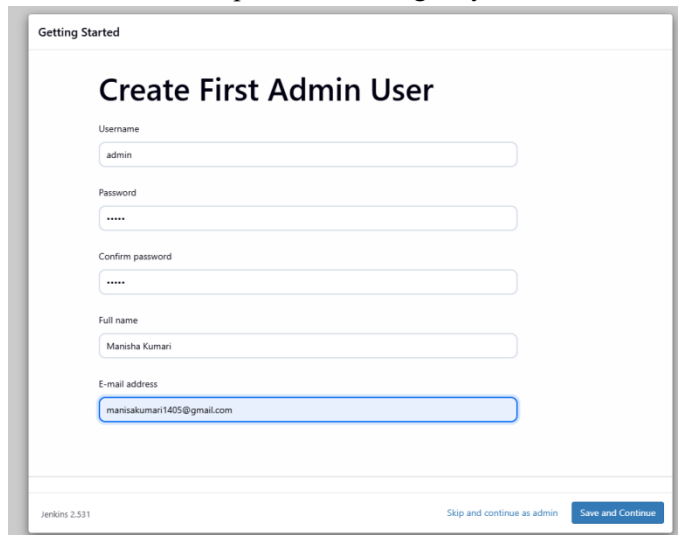
Getting Started

Folders	Formatter	Ant	Gradle
Timestamp	Workspace Cleanup	Ant	Gradle
Pipeline	GitHub Branch Source	Pipeline: GitHub Groovy Libraries	Pipeline Graph View
Git	SSH Build Agents	Matrix Authorization Strategy	LDAP
Email Extension	Mailer	Dark Theme	

** - required dependency

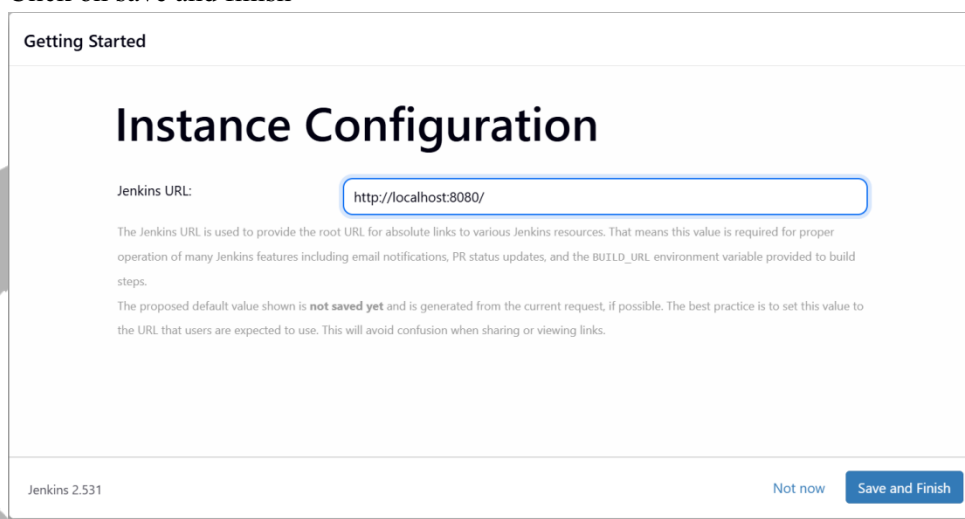
Jenkins 2.531

19. Then un -> admin, pwd -> admin, give your full name and email id & click on save & continue



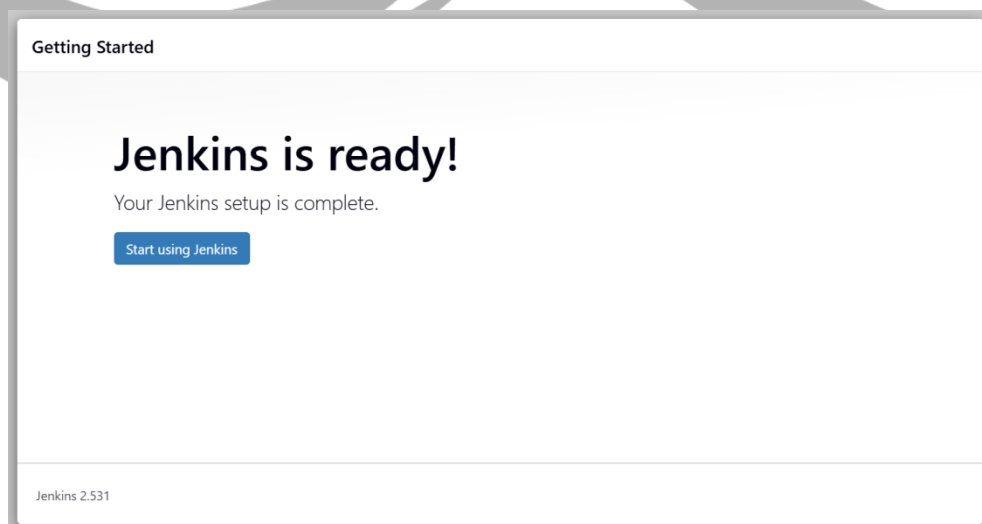
The screenshot shows the 'Create First Admin User' form in Jenkins. The form has the following fields: Username (admin), Password (****), Confirm password (****), Full name (Manisha Kumari), and E-mail address (manisakumari1405@gmail.com). At the bottom, there are two buttons: 'Skip and continue as admin' and 'Save and Continue'.

20. Click on save and finish



The screenshot shows the 'Instance Configuration' page in Jenkins. The 'Jenkins URL' field is set to 'http://localhost:8080/'. Below the field, there is a paragraph explaining the Jenkins URL and its importance. At the bottom, there are two buttons: 'Not now' and 'Save and Finish'.

21. And all set !!!



The screenshot shows the 'Jenkins is ready!' page in Jenkins. The page has the heading 'Jenkins is ready!' and the text 'Your Jenkins setup is complete.' Below this, there is a button labeled 'Start using Jenkins'.

a. **Precondition 1:** Install Plugins

- a. Navigate to:
Jenkins → Manage Jenkins → Manage Plugins → Select "Available"
- b. Search and install the following plugins:
 - Maven Integration Plugin
 - GitHub Integration Plugin

b. **Precondition 2:** Configure Environment Variables

- b. Navigate to:
Jenkins → Manage Jenkins → Global Tool Configuration



Tools

Configure tools, their locations and automatic installers.

c. Set the following paths:

- **JDK:** C:\Program Files\Java\jdk-21

JDK installations ^ Edited

Add JDK

JDK

Name
JDK

JAVA_HOME
C:\Program Files\Java\jdk-17

☐ Install automatically ?

Add JDK

- **Maven:** D:\apache-maven-3.9.10-bin\apache-maven-3.9.10

Maven installations ^ Edited

Add Maven

Maven

Name
MVN_HOME

MAVEN_HOME
C:\Program Files\apache-maven-3.9.9

☐ Install automatically ?

- **Git:** C:\Program Files\Git\bin\git.exe
<https://github.com/git-for-windows/git/releases/download/v2.51.0.windows.2/Git-2.51.0.2-64-bit.exe>

Git installations

Git

Name
Default

Path to Git executable ?
C:\Program Files\Git\bin\git.exe

☐ Install automatically ?

1. Use of Jenkins in Testing

Jenkins is a Continuous Integration (CI) tool that monitors framework builds in Git and triggers actions based on the changes.

2. Advantages of Jenkins

1. **Three Execution Levels:**

- a. **On Demand:** Start execution manually based on user request.
- b. **On Schedule:** Automatically trigger execution at scheduled times.
- c. **Poll SCM:** Continuously monitor the SCM (e.g., GitHub) and trigger execution upon new builds or test scripts.

2. **Email Notifications:**

- a. Sends execution reports upon completion.

3. **Alerts users** if builds break due to compilation errors.

4. **Runtime Parameters:**

- a. Supports dynamic input during job execution.

5. **Pipeline Jobs:**

- a. Enables sequential execution of multiple jobs.
-

3. How to Create a Job in Jenkins

1. Need Source Code Management data

- a. url
- b. username
- c. token => password

1. **Log in** to Jenkins.

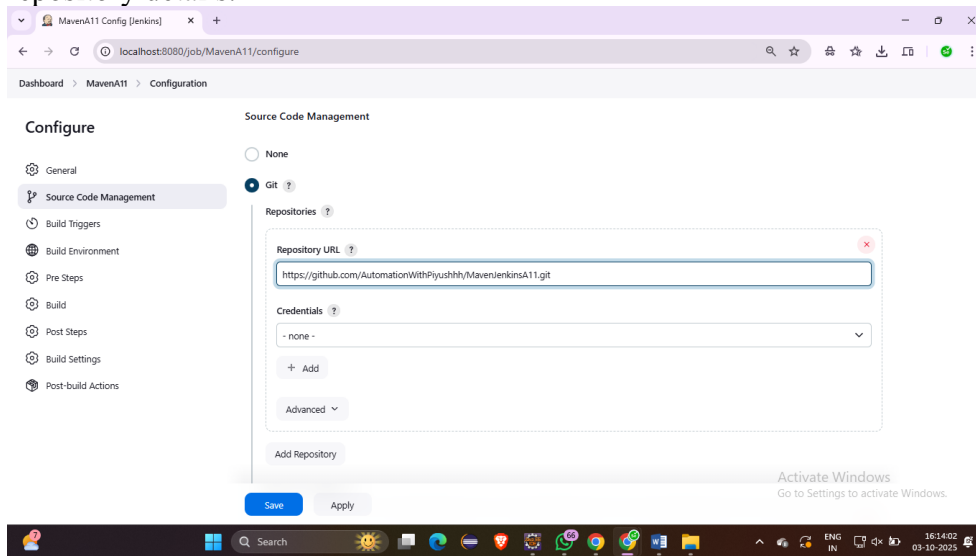
2. Click **New Item**.

3. Select **Maven Project** and enter a **job name**.

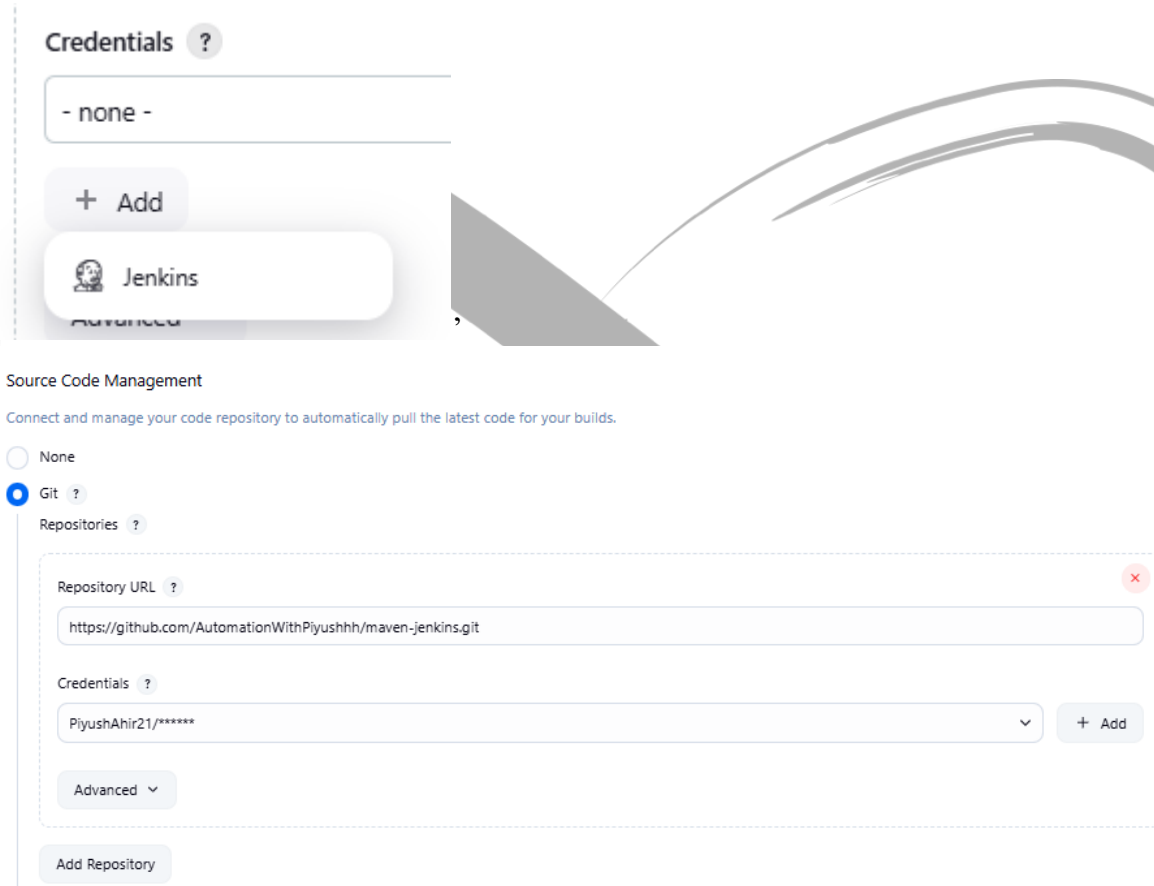
4. Click **OK**.

4. Configure item

1. Don't give anything to general and move to Source Code Management, select **Git** and enter repository details.



2. Add credentials



3. In triggers, select only first

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☒ Build whenever a SNAPSHOT dependency is built ?
- ☐ Schedule build when some upstream has no successful builds ?
- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ GitHub Branches
- ☐ GitHub Pull Requests ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Activate Windows

4. Select first in environment to remove previous workspace

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

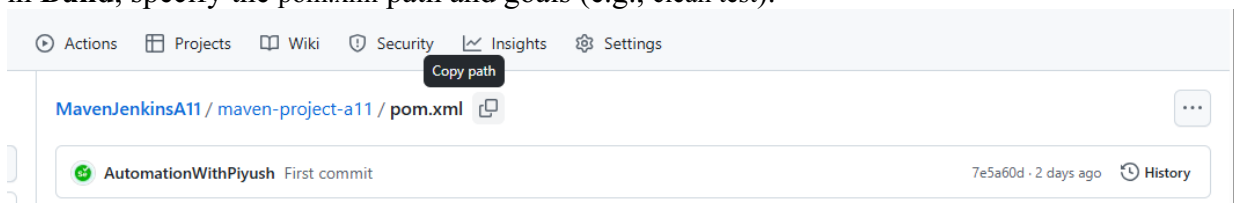
- ☒ Delete workspace before build starts
- ☐ Use secret text(s) or file(s) ?
- ☐ Provide Configuration files ?
- ☐ Add timestamps to the Console Output
- ☐ Inspect build log for published build scans
- ☐ Terminate a build if it's stuck
- ☐ With Ant ?

5. Don't select anything for pre steps

Pre Steps

Add pre-build step ▾

6. in **Build**, specify the pom.xml path and goals (e.g., clean test).



so it should be like this

Build

Root POM ?

maven-project-a11/pom.xml

Goals and options ?

test

7. post steps

Post Steps

- ☐ Run only if build succeeds
- ☐ Run only if build succeeds or is unstable
- ☒ Run regardless of build result
Should the post-build steps run only for successful builds, etc.

Add post-build step ▾

8. build setting and post build actions

Build Settings

- ☐ E-mail Notification

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Add post-build action ▾

9. Click **Save**.



Jenkins / maven-jenkins



Status



Changes



Workspace



Build Now



Configure



Delete Maven project



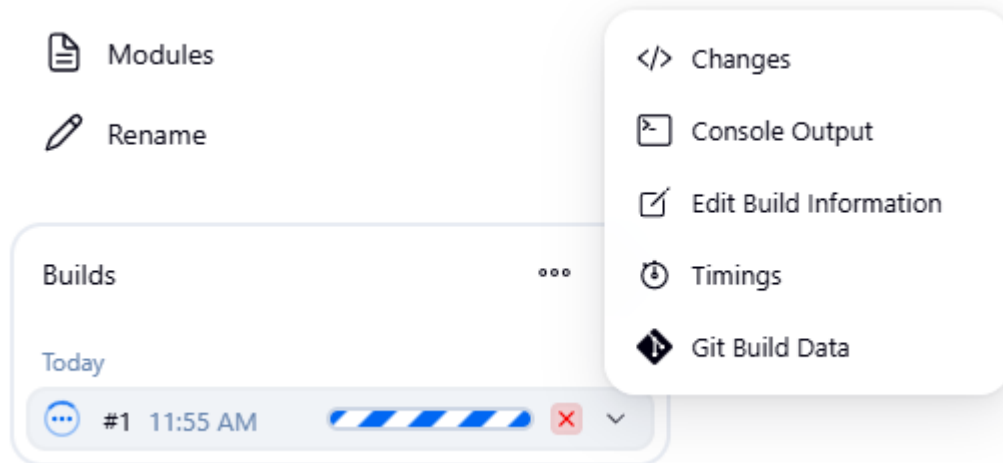
Modules



Rename

10.

11. Click on Build Now and it will start executing

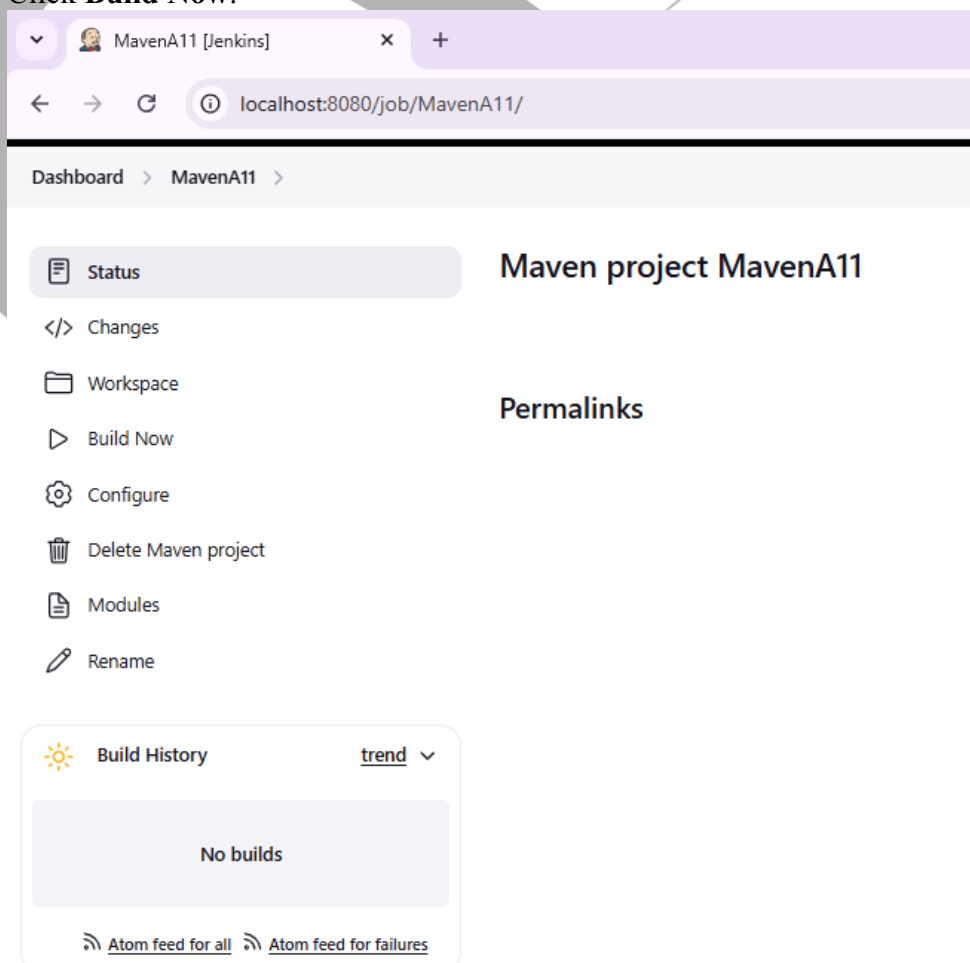


12. Click on dropdown and then console output to check the results

13.

5. Executing a Jenkins Job

1. Log in to Jenkins.
2. Open the job.
3. Click **Build Now**.



6. Viewing Job Results

1. Navigate to the job.
 2. Under **Build History**, click the latest build.
 3. Select **Console Output** to view execution logs.
-

7. Scheduling a Job

1. Navigate to the job's **Configure** page.
2. Check **Build Periodically**.
3. Enter a schedule (e.g., * 2 * * * for daily at 2 AM).
 - **Format:** MINUTE HOUR DOM MONTH DOW
 - **MINUTE:** 0-59
 - **HOUR:** 0-23
 - **DOM:** 1-31 (Day of Month)
 - **MONTH:** 1-12
 - **DOW:** 0-7 (0 and 7 = Sunday)
 - 0 – Sunday
 - 1 – Monday
 - 2 – Tuesday
 - 3 – Wednesday
 - 4 – Thursday
 - 5 – Friday
 - 6 – Saturday
 - 7 – Sunday

8. Configuring Poll SCM

1. Navigate to the job's **Configure** page.
2. Check **Poll SCM**.
3. Enter a schedule as * * * * *

BEHAVIOUR DRIVEN DEVELOPMENT

What is BDD?

- **BDD (Behaviour Driven Development)** is a software development process used in **Agile** methodology.
 - It is an **iterative** approach that bridges the gap between **technical** and **non-technical** team members.
-

Iteration Phases in BDD

1. **Discovery** – Finding out *what the system could do*.
 2. **Formulation** – Deciding *what the system should do*.
 3. **Automation** – Implementing *what the system actually does*.
-

TDD Process (Cycle)

Test Creation → Test Fails → Code → Refactor → Pass

- Write a test that describes desired behaviour.
 - Run test → It fails (as code doesn't exist yet).
 - Write minimal code to pass the test.
 - Refactor and repeat.
-

BDD vs TDD

- **TDD** focuses more on **coding and technical validation**.
 - **BDD** focuses on **behaviour, collaboration**, and shared understanding across the whole team.
 - BDD extends TDD with **human-readable tests** written in **Gherkin language**.
-

Introduction to Cucumber and Gherkin

What is Cucumber?

- **Cucumber** is a **BDD framework** that helps automate user acceptance tests.
 - It acts as a **bridge between business and technical teams**.
 - **Cucumber alone** doesn't perform automation—it needs to be combined with **Selenium + Java**.
-

Popular BDD Tools

1. Cucumber
2. JBehave
3. Behave
4. SpecFlow (.NET)

Gherkin Language

- A **parsable, plain-text language** used to write **feature files**.
- Easy to understand for both technical and non-technical stakeholders.
- Uses **keywords** to describe test scenarios.

Gherkin Keywords

Keyword	Meaning	Example
Given	Preconditions	“Given browser is opened”
When	Action	“When logs in with valid credentials”
Then	Expected outcome	“Then user should be logged in successfully”
And	Used for concatenation	“And browser should close”

Example User Story

Feature: Login

Scenario: Validate user login with valid credentials

Given Browser is opened and URL is navigated

And Login page is displayed

When User enters valid username and valid password

And User clicks on login button

Then User should be successfully logged in

And Browser should close

Step Definitions in Cucumber

What is Step Definition?

- Step definitions are the **glue code** that connects **Gherkin steps** to **actual automation code (Selenium + Java)**.
- Every step written in the feature file must have a corresponding **method** in the step definition file.

Project Structure

```
src/test/java
├── stepDefinitions
│   └── LoginSteps.java
├── runner
│   └── TestRunner.java
src/test/resources
├── features
│   ├── login.feature
│   └── addToCart.feature
```

Required Dependencies

1. **Cucumber Core** – Provides APIs for Cucumber framework.
2. **Cucumber Java** – Integrates Cucumber with Java.
3. **Cucumber TestNG / JUnit** – Runs test cases.
4. **Selenium Java** – For web automation.

(All dependencies are added in pom.xml when using Maven.)

Components of Cucumber Framework

Component	Description
Feature File	Contains scenarios in Gherkin format. Each feature represents a functionality.
Step Definition File	Contains Java methods implementing Gherkin steps using Selenium code.
Runner Class	Executes the feature file using JUnit or TestNG, and generates reports.

Execution Flow

1. **Feature file** is read →
 2. **Cucumber** identifies step keywords (Given, When, Then) →
 3. **Step Definition file** provides Java implementation →
 4. **Runner class** executes all mapped steps →
 5. **Reports** are generated.
-

Advantages of BDD

- Promotes **collaboration** among Developers, QA, and Business Analysts.
 - Test cases are **easy to read and maintain**.
 - Improves **requirement understanding** through real examples.
 - Ensures **better coverage** and transparency.
-

Disadvantages of BDD

- Not suitable for all projects (especially small or purely technical ones).
 - Involves **extra setup and annotations**, making it heavier than TestNG for simple tests.
 - Requires **discipline and communication** among all stakeholders.
-

Summary of a BDD Project with Selenium

1. Stakeholders write **feature file** using Gherkin.
 2. Cucumber generates **step definitions** automatically.
 3. Automation testers write **Selenium code** inside those step methods.
 4. **Runner class** (JUnit/TestNG) executes scenarios and generates reports.
-

Typical Runner Class (Example)

```
@CucumberOptions(features = "src/test/resources/features",  
                  glue = { "steps", "hooks" },  
                  tags = "@smoke")  
public class TestRunner extends AbstractTestNGCucumberTests {}
```

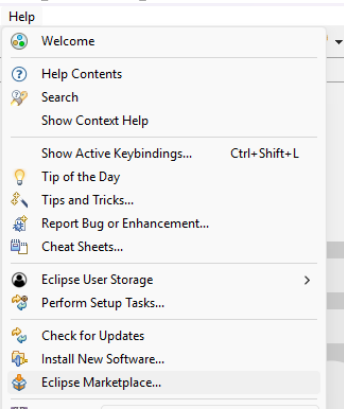
In short:

- **BDD** = “Define Behaviour, Then Develop.”
- **Cucumber** = “Tool to implement BDD.”
- **Gherkin** = “Language to write BDD scenarios.”
- **Step Definition** = “Glue between Gherkin and Selenium.”
- **Runner** = “Executor for your BDD tests.”

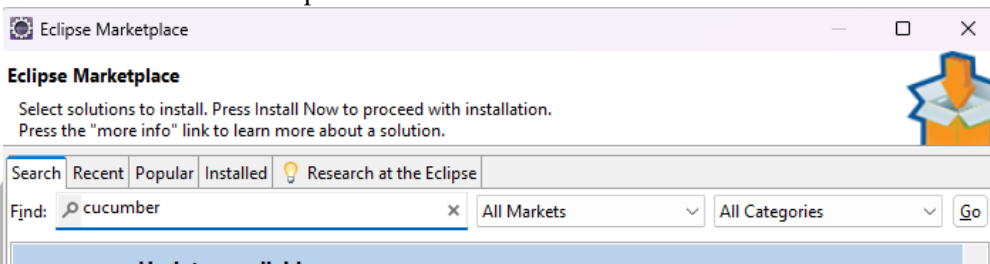
CUCUMBER

How to integrate Cucumber?

1. Help -> eclipse MarketPlace



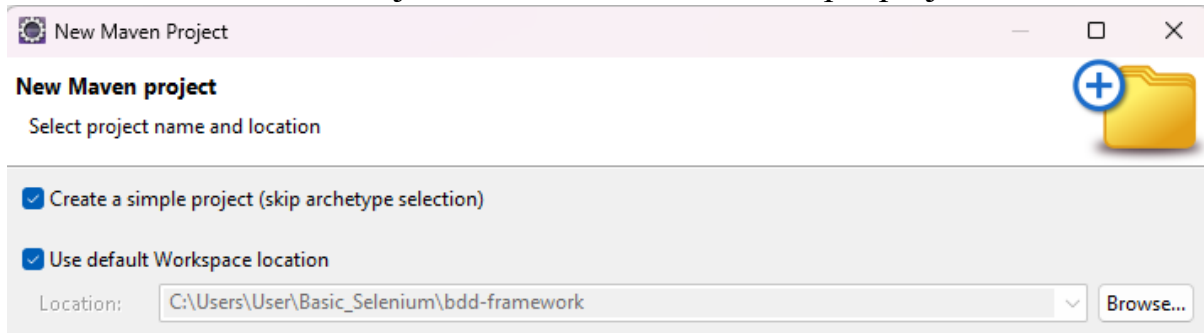
2. Search for cucumber and press enter



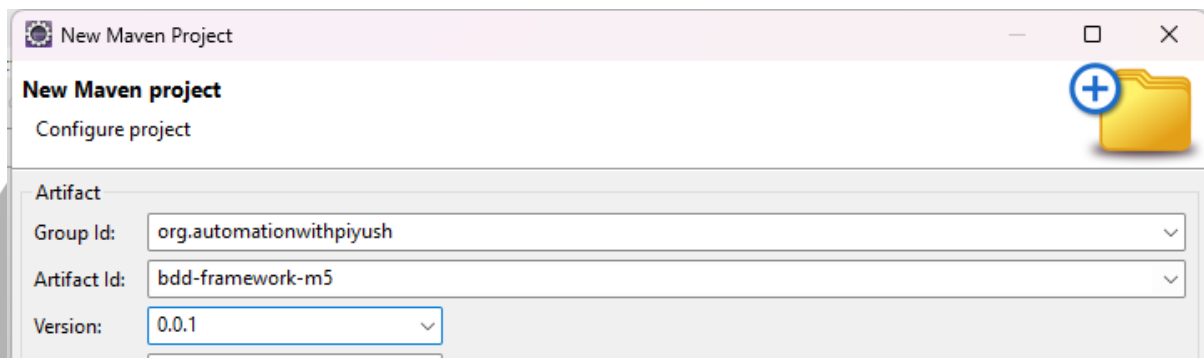
3. Click on install near cucumber plugin
4. It will ask for trust certificates, there select all and trust selected
5. Lastly restart now

How to create Maven project and give dependency?

1. File -> new -> Maven Project -> check on create a simple project => click on next



- 2.
3. Give
group id => org.automationwithpiyush
artefact id => bdd-framework-(your batch code)
version => 0.0.1
and click on finish



4. Open pom.xml and give these dependencies

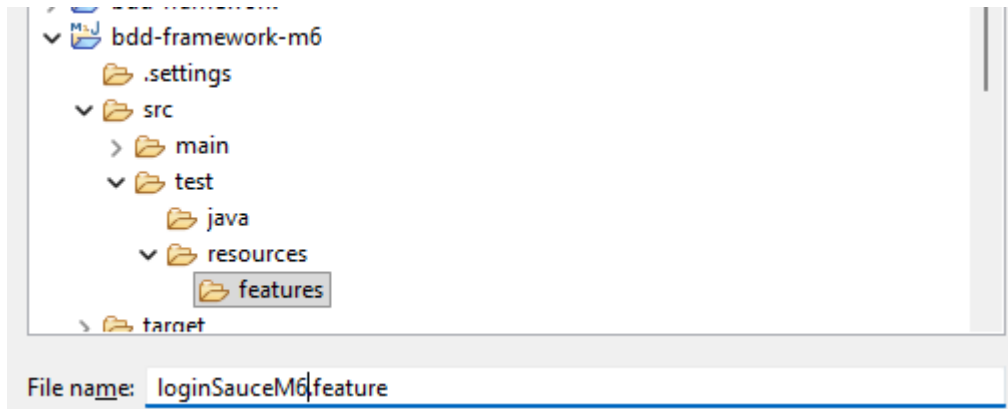
```
<dependencies>

    <dependency>
        <groupId>io.cucumber</groupId>
        <artifactId>cucumber-java</artifactId>
        <version>LATEST</version>
    </dependency>

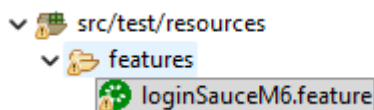
    <dependency>
        <groupId>io.cucumber</groupId>
        <artifactId>cucumber-testng</artifactId>
        <version>LATEST</version>
    </dependency>
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>LATEST</version>
    </dependency>
</dependencies>
```


How to create & run a cucumber feature file?

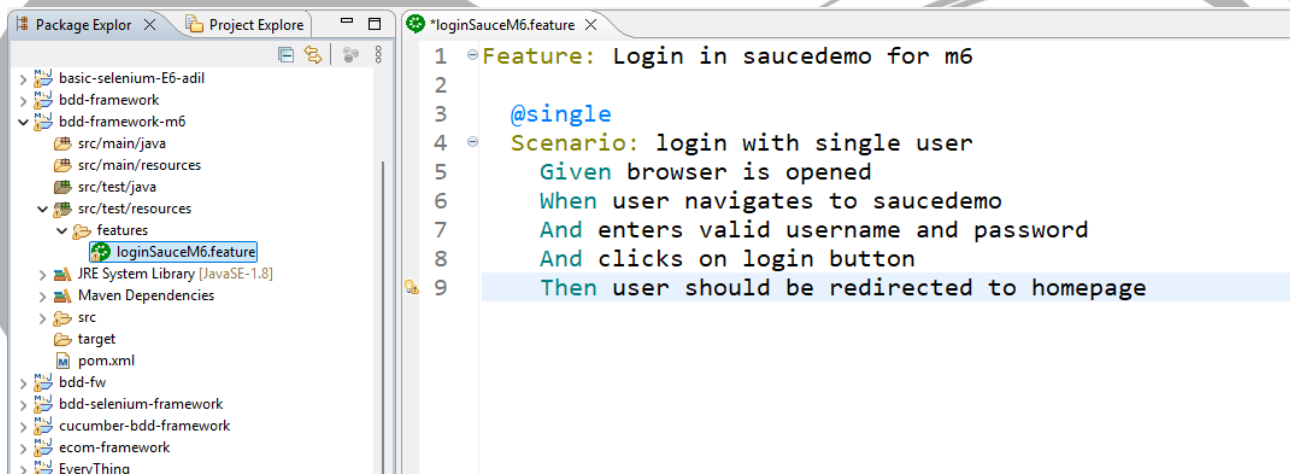
1. Create a folder in src/test/resources => features
2. Create a .feature file inside features folder



Result:



3. Write code like this



4. Right click => run as => cucumber feature

5. Go to console and copy the snippets

@single

Scenario: login with single user #
src/test/resources/features/loginSauceM6.feature:4

- Given browser is opened
- ~ When user navigates to saucedemo
- ~ And enters valid username and password
- ~ And clicks on login button
- ~ Then user should be redirected to homepage

Undefined scenarios:

1) login with single user # file:///C:/Users/User/Basic_Selenium/bdd-framework-m6/src/test/resources/features/loginSauceM6.feature:4

1 scenarios (1 undefined)

5 steps (4 skipped, 1 undefined)

0m 1.479s

You can implement missing steps with the snippets below:

```
@Given("browser is opened")
public void browser_is_opened() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}

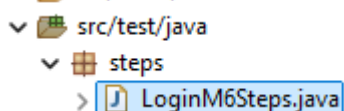
@When("user navigates to saucedemo")
public void user_navigates_to_saucedemo() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}

@When("enters valid username and password")
public void enters_valid_username_and_password() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}

@When("clicks on login button")
public void clicks_on_login_button() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}

@Then("user should be redirected to homepage")
public void user_should_be_redirected_to_homepage() {
    // Write code here that turns the phrase above into concrete actions
    throw new io.cucumber.java.PendingException();
}
```

6. Create a package in src/test/java => steps
7. Create one step definition file inside steps package



8. Give implementation to these steps one by one

```
public class LoginM6Steps {

    WebDriver driver;

    @Given("browser is opened")
    public void browser_is_opened() {
        driver = new EdgeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(15));
    }

    @When("user navigates to saucedemo")
    public void user_navigates_to_saucedemo() {
        driver.get("https://www.saucedemo.com/");
    }

    @When("enters valid username and password")
    public void enters_valid_username_and_password() {
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
    }

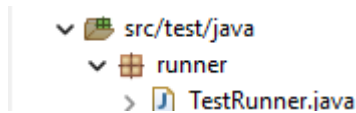
    @When("clicks on login button")
    public void clicks_on_login_button() {
        driver.findElement(By.id("login-button")).click();
    }

    @Then("user should be redirected to homepage")
    public void user_should_be_redirected_to_homepage() {
        Assert.assertTrue(driver.getCurrentUrl().contains("inventory"), "Could not redirected to home page");
    }

    @And("browser is closed")
    public void browser_is_closed() {
        driver.quit();
    }
}
```

How to create & run a TestRunner?

1. Create a TestRunner.java inside runner package



2. Copy paste this code

```
package runner;

import io.cucumber.testng.AbstractTestNGCucumberTests;
import io.cucumber.testng.CucumberOptions;

@CucumberOptions(
    features = "src/test/resources/features",
    glue = "steps "
)
public class TestRunner extends AbstractTestNGCucumberTests {}
```

Right click => Run as => TestNG test

BaseTest:-

```
package base;

import java.time.Duration;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;

public class BaseTest {
    public static WebDriver driver;

    public void setUp() {
        driver = new EdgeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(15));
    }

    public void tearDown() {
        driver.quit();
    }
}
```

Hooks:-

```
package hooks;

import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;

import base.BaseTest;
import io.cucumber.java.After;
```

```

import io.cucumber.java.Before;
import io.cucumber.java.Scenario;

public class MyHooks extends BaseTest{
    @Before
    public void beforeScenario() {
        setUp();
    }

    @After
    public void tearDown(Scenario scenario) {
        if (scenario.isFailed()) {
            String screenshotName = scenario.getName().replaceAll(" ", "_");

            byte[] ss = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BYTES);
            scenario.attach(ss, "image/png", screenshotName);
        }

        tearDown();
    }
}

```

